

Review paper · Database Management Systems

Agent memory as a database problem

A review of Notion's Lore, and what happens when its schema is given a database to run on.

Harshit Khemani · Kush Ahuja · Madhav Bassi · Kushagra Agrawal

Submitted to Dr. Poonam Sangwan · BTech CSE coursework

Subject: github.com/makenotion/lore at 95c3558 (1.0.0, MIT, Notion Labs, Inc.), read 2026-08-04.

Measured 22 August 2026.

Everything, downloadable

The paper	PDF · A4 · typeset from this page
The paper	ePub · reflows on an e-reader
The talk	PDF · 16:9 · one slide per page
The talk, live	HTML · arrow keys, or swipe
The dataset	JSON · every number in the paper



<https://dbms-memory.khe.money>

Abstract

Lore is Notion's open-source memory system for AI agents. It gives an assistant a persistent, shared vault so that context survives a cleared session, a new branch, or a handover between people, and it does so by storing that memory as rows in five Notion databases: Projects, Topics, Memories, Entities and Facts. The design is unusual and deliberate. The memory is human-readable, editable by hand, and inherits the permissions of a tool the team already pays for; there is no database to run.

This paper reviews that design as a database design. We reconstruct Lore's schema from its source — 80 properties across five databases, read from `src/notion/schema.ts` at commit 95c3558 — and analyse it with the ordinary tools: the entity-relationship model, normal forms, functional dependencies, the temporal-database distinction between valid time and transaction time, and the invariants a schema can declare. We then reimplement the same schema on SQLite and PostgreSQL, generate a vault of 11 885 memories and 4 277 facts whose ground truth is definitional rather than judged, and put a workload of 361 questions in 10 classes through three stores: an emulator restricted to the documented Notion Data API, and the two engines.

The result is not that the vault gets things wrong. It gets them right: the Notion arm answers **100.0%** of the workload exactly, the same as both engines. What separates them is what the answer costs and what the store will refuse. Answering the workload takes the Data API **522 686** requests against **361** SQL statements — 48.4 h of wall-clock floor at the documented 3 requests per second, before anyone's network is involved. A single join from a claim to the memory that supports it costs 1 138 requests (1 138×). Searching what memories actually say costs 11 892 (66.1 min), because the search endpoint matches titles and a memory's text is page blocks.

The sharper finding is about refusal. Eight agents upserting the same topic key lose **87.1%** of their updates, because the read-then-write protocol has no conditional write to close; the same workload through a unique index loses none. PostgreSQL

declines even to *add* a temporal exclusion constraint to the vault as generated, because 70 subject-predicate pairs already assert two different objects over overlapping time — contradictions that Lore detects afterwards, with a documented scan cap, and that a range-typed constraint refuses at the point of writing. Three normalisation violations ship, each for a substrate reason we can name: a repeating group in one cell, a stored expression standing in for an index Notion has no way to create, and a Boyce-Codd violation in the centre of the knowledge graph that leaves 105 facts whose title and whose relation disagree about their own subject.

We argue that these are not defects so much as the visible price of a real trade, and that the trade is legible: Lore exchanges every guarantee a database declares — uniqueness, referential integrity, atomicity, the ability to state a join — for human-legible memory, zero infrastructure and inherited permissions, and then re-implements the discarded guarantees as scanners, migrations and filesystem locks. Its own documentation is unusually candid about where the seams are. The contribution here is to put numbers on them.

100% workload answered exactly, all three stores	522 686 requests the Data API needs
361 SQL statements the same workload needs	87% updates lost by 8 concurrent writers

1. Introduction

An agent that forgets is a tool. An agent that remembers is a colleague, and colleagues need somewhere to keep what they know. The last three years have produced a shelf of answers to that need — paged context windows ^[3], memory streams with reflection ^[9], extraction-and-update pipelines over a vector store ^[5], temporal knowledge graphs ^[6], and, at the simple end, a markdown file the model rewrites ^[15]. What almost none of them do is take seriously the possibility that this is a problem the database community solved, in stages, between 1970 and 2011.

Lore is interesting because it comes closer than most. It is not a vector index with a metadata blob bolted on. It is a schema: five related tables, a subject-predicate-object fact relation with validity intervals and a confidence, a canonical entity registry with aliases, and a set of services that maintain them. Reading its source, one keeps meeting old friends — a specialisation hierarchy, a bridge table, a derived attribute, a supersession chain — and the pleasure of the exercise is watching a team arrive at the relational model from the outside, under a constraint the relational model has never had to work under: the store is a document database exposed over HTTP, and it has no joins, no aggregates, no unique indexes, no constraints, and no conditional writes.

This paper asks a narrow question with a broad answer. *What does that constraint actually cost?* Not rhetorically — in requests, in bytes, in rows the client has to look at itself, and in states the store will happily accept and a database would refuse.

Our contributions are four.

A schema reconstruction. Lore's five databases, restated as a relational schema with an entity-relationship diagram, functional dependencies, and a normalisation walk. Every property name and enumeration is read from `src/notion/schema.ts`, not from prose, and the diagram, the DDL and the argument are all generated from one declaration so they cannot drift (§4, §5).

A workload with non-circular ground truth. 10 question classes read off Lore's own commands and hooks, each annotated with the minimum relational algebra it needs, over a vault generated world-first so that the correct answer is definitional (§7).

A cost measurement in substrate-independent units. Round trips, bytes and client-side rows, against an emulator restricted to the documented Data API, with every restriction carrying the reference page it comes from (§8).

Integrity experiments. Four invariants, attacked deliberately, on both the vault and a reimplementation that can declare them (§9).

We are reviewing a system, not competing with it. Where a measurement came out smaller than the argument would have liked, §9.5 and §14 say so.

2. Background

2.1 What an agent memory has to do

The surveys converge on a decomposition. Du and colleagues split memory operations into six atoms — consolidation, updating, indexing, forgetting, retrieval and compression ^[19] — and every one of the six has a database name. Consolidation is materialisation. Updating is the update anomaly problem. Indexing is indexing. Forgetting is a retention policy. Retrieval is query processing. Compression is, roughly, summarisation as a materialised view. Zhang and colleagues' earlier survey ^[17] and Wu and colleagues' human-memory mapping ^[18] reach the same place from different directions.

What the benchmarks then show is that the hard part is not recall of a single fact but recall *through change*. LongMemEval isolates five abilities and two of them — temporal reasoning and knowledge updates — are exactly the ones a validity interval exists to serve ^[24]; LoCoMo finds that models struggle with long-range temporal and causal dynamics even when the whole history is in context ^[23]. An agent memory that cannot say *when* something was true is not a memory, it is a pile.

2.2 Why this is a database problem

Codd's 1970 paper is about the same complaint ^[39]: applications were storing data in a shape that made some questions easy and others unaskable, and changing the shape broke the applications. Chen gave the modelling vocabulary six years later ^[41]. Temporal databases gave us the distinction that the agent-memory literature keeps rediscovering — valid time, when a fact was true in the world, against transaction time, when the database came to believe it ^[42]^[43] — and SQL:2011 standardised it ^[44]. Record linkage has a theory from 1969 ^[46] and a generic framework from 2009 ^[47]. Provenance has a characterisation ^[48] and a survey ^[49]. The lost update is in Gray ^[51] and the isolation levels that permit it are in Berenson ^[52].

None of this is a criticism of the agent-memory literature, which is solving a different and newer problem. It is an observation that a memory system arriving at subject-predicate-object triples with validity windows and a confidence score has arrived at RDF ^[53] with named graphs ^[55] and a probabilistic layer ^[58], and that the questions those communities asked next are the questions worth asking here.

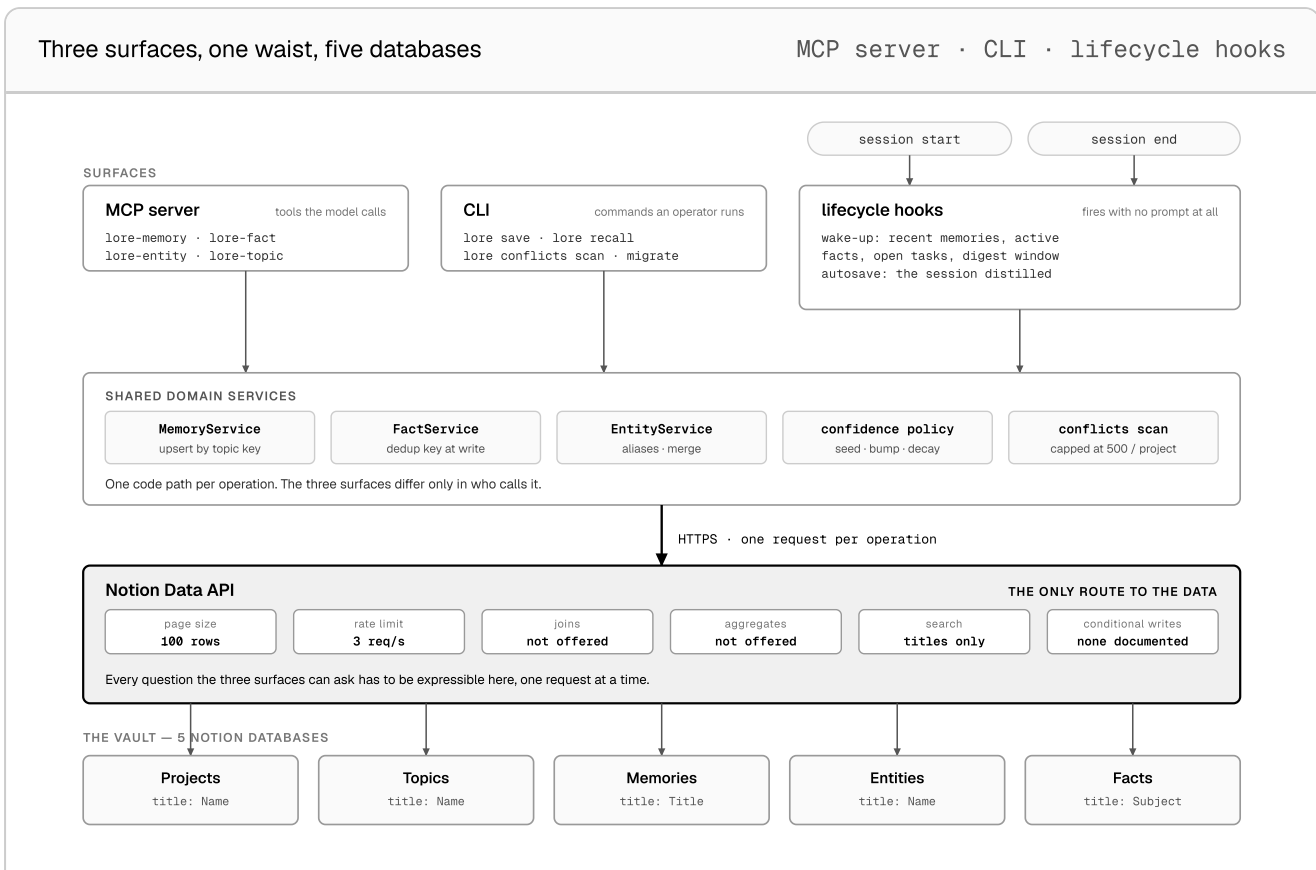
2.3 The Model Context Protocol

Lore reaches assistants through MCP, opened by Anthropic in November 2024 ^[25]. The specification defines exactly three server primitives — resources, prompts and tools ^[26]. Lore exposes its entire vault through tools alone, six of them, each multiplexing several actions behind one registration. That is a substantive design choice: a vault modelled as resources would be addressable and cacheable by the host, and a vault modelled as tools is a set of procedures the model must decide to call.

3. The system under review

Lore is an MIT-licensed Node package, @notionhq/lore, at version 1.0.0 when we read it. Three surfaces share one set of domain services: an MCP server for assistants, a CLI for people, and lifecycle hooks that load context at session start and save it at session end. Underneath all three is the Notion Data API, and underneath that is a page: a vault is one Notion page containing five databases.

Figure 1 — Lore's surfaces



Layers, not a call graph: an arrow means the layer above can only reach the layer below through this edge.

Highlighted: the Notion Data API, the single point every question has to be expressible at. The two chips above the hooks are the events that fire them — session start and session end — and neither involves a prompt, so a session pays the wake-up bill before the agent has said anything and the autosave bill after it has finished. Property names and the five databases are read from Lore's own schema; the service names are the modules those operations live in.

Every operation Lore performs — whether the model called a tool, an operator typed a command, or a hook fired on its own — descends through the same domain services onto the same HTTP surface. That surface is where the relational vocabulary stops: it offers no join and no aggregate, returns at most 100 rows per request, and is documented at 3 requests per second. Section 3 is about what that waist costs; Sections 5 and 6 are about what it makes undeclarable.

Three entry points, one set of services, one substrate. The hooks are what make the memory ambient rather than requested: wake-up fires before the assistant's first response, autosave after its last.

The choice of substrate is the whole design. Notion is a document store with a relational veneer: a database is a collection of pages, a page has typed properties, and a relation property holds an array of page identifiers. What it is not is a query processor. The public reference documents filters and sorts over properties, cursor pagination at a hundred rows a request, and a search endpoint; it offers no join, no aggregate, no unique index, no check constraint, and no conditional write.

Table 1 — The documented limits our emulator enforces

Constraint	Documented value	Reference
Rows per request	10 default, 100 maximum	intro
Request rate, one connection	3 per second on average	request-limits
rich_text value	2 000 characters	request-limits
Relation property	100 related pages	request-limits
multi_select property	100 options	request-limits
Request payload	500.0 kB	request-limits
Joins across databases	not offered	post-database-query
Aggregate functions	not offered	post-database-query
Search endpoint matches	titles only	post-search
Conditional writes	none documented	intro

Each row is a constraint the Notion arm respects, with the reference page it comes from. Nothing in the emulator refuses anything the reference offers, and nothing permits anything it does not.

Two of these deserve emphasis because the paper returns to them. First, **the search endpoint matches titles**, not page content. A memory in Lore is a title, a set of properties, and a body of blocks; the body is where the sentence somebody wrote lives, and no property filter can see it. Second, **there is no documented conditional write** — no ETag, no If-Match, no version token — so a client that reads a row and then writes it cannot ask the server to apply the write only if nothing changed in between.

4. The data model

Lore's schema is declared in one TypeScript file that describes itself as “the single source of truth for every read and write on this database”. We transcribed it into the relational vocabulary, and everything downstream in this repository — the diagram below, the DDL both engines execute, the property catalogue the emulator creates, and the dependency analysis in §5 — is generated from that one transcription.

Table 2 — The five databases

Read from `src/notion/schema.ts` at commit 95c3558.

Database	Title property	Properties	Relations	Derived	System-managed
Projects	Name	6	0	0	1
Topics	Name	4	1	0	1
Memories	Title	39	3	0	7
Entities	Name	7	2	0	1
Facts	Subject	24	5	3	6

Property counts are computed from the declaration, not counted by hand. 'Derived' means a column whose value is a function of other columns in the same row; 'system-managed' means a column the agent-facing tools do not accept.

Three things about that table are worth stopping on.

Memories is a single-table specialisation with ten subtypes. The Kind property takes ten values — note, decision, incident, runbook, postmortem, policy, state, operational, task and procedure — and the subtypes do not share attributes. A task has Task State, Blocked By and Done At; a decision has Alternatives, Consequences and Decided At; a pinned block has Pinned Priority and Mutability. This is the classic single-table specialisation, and it is paid for exactly as the textbooks say: in columns that are null for most rows, and in the impossibility of declaring “a task must have a task state” anywhere the store can enforce it.

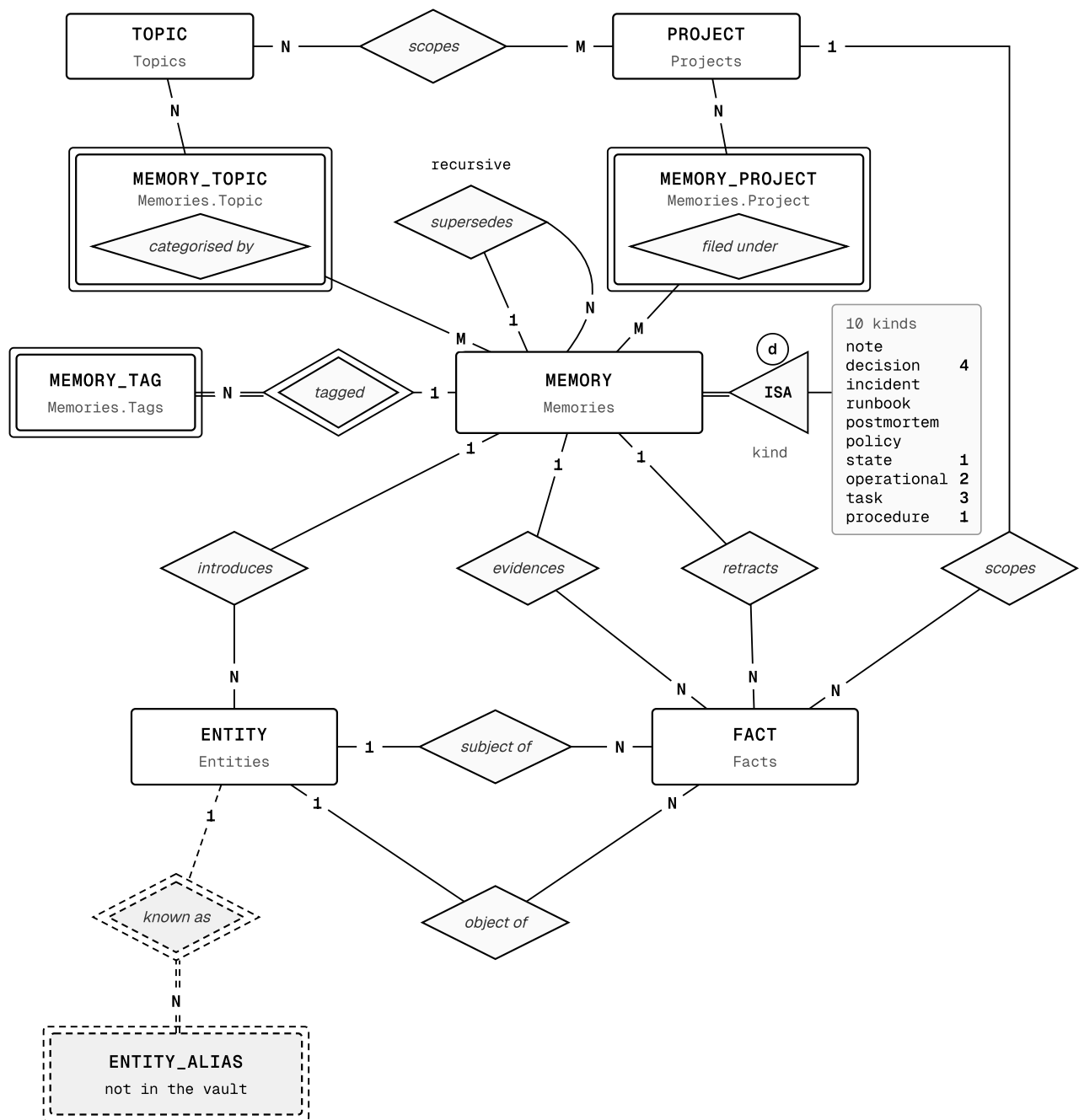
Facts is a triple store with two time axes and two confidences. Subject, predicate and object; Valid From and Valid Until; Observed At, Invalidated At and Invalidated By; a categorical Confidence and a numeric Confidence Score. Lore's comment on the four dates is precise about what they are for:

“Valid From / Valid Until model domain truth (when the fact was true in the world); Observed At and Invalidated At model what Lore knew and when. Together they implement the bitemporal axis used for as-of recall.”

`src/notion/schema.ts`

Facts is also the link table. Four of Lore's predicates are referential plumbing rather than domain knowledge: mentions is auto-emitted when a memory is saved, decided_by, supersedes_decision and informs are written only by the decision service. They live in the same relation as owned_by and depends_on, so any aggregate over “the facts we know” counts pointers as claims unless it filters them out. In our generated vault those system predicates are 4.8% of all facts.

Figure 2 — Entity-relationship diagram, Chen notation



Rectangle = entity, one Notion database · double rectangle = bridge relation, with no identity apart from its parent · a rectangle around a diamond = the bridge that resolves an M:N · dashed and in the accent = exists only in the reimplementations (ENTITY_ALIAS, the shape a normalised schema would give the Aliases cell) · diamond = relationship · double diamond = identifying · double line = total participation · M / N / 1 are read at the end nearest each entity · ISA with (d) = disjoint and total specialisation on kind; the number beside a kind is how many properties exist for that kind alone.

Generated from the schema declaration. Every box is a relation the reimplementations create; every edge is a foreign key it enforces. The dashed box is ENTITY_ALIAS — the relation a normalised schema would have and the vault does not, because Lore stores aliases as a joined string in one cell.

4.1 The property catalogue

A reader with their own vault can check our reconstruction against it. The catalogue below lists every property we model, its Notion type, and the flags that matter for the analysis: whether it is derived, whether it is system-managed, whether it is a repeating group inside a single cell, and which temporal axis it belongs to. Properties marked *not modelled* exist in Lore and are left null by our generator because the workload does not exercise them; naming them is cheaper than a footnote and harder to forget.

Table 3 — Property catalogue

PROJECT · Projects · A named scope. Directories map onto projects by longest-prefix match. 6 columns in the reimplementation: 0 relations, 0 derived, 1 system-managed, 0 declared but left null by the vault generator.

Property	Lore name	Notion type	Flags	Note
<u>project_id</u>	—	page id	system-managed	
name	Name	title cap 2000	title	
type	Type	select 3 options	—	
path	Path	rich_text cap 2000	—	
status	Status	select 2 options	—	
description	Description	rich_text cap 2000	—	

Underlined = primary key · → = foreign key in the reimplementation, a relation property in the vault · cap = the limit the Notion API enforces on that type · every name in the two middle columns was read from src/notion/schema.ts at commit 95c3558.

TOPIC · Topics · A category. The Project relation is dual_property, so a topic may span projects. 4 columns in the reimplementation: 1 relation, 0 derived, 1 system-managed, 0 declared but left null by the vault generator.

Property	Lore name	Notion type	Flags	Note
<u>topic_id</u>	—	page id	system-managed	
name	Name	title cap 2000	title	
description	Description	rich_text cap 2000	—	
project_id → project.project_id	Project	relation cap 100	—	

Underlined = primary key · → = foreign key in the reimplementation, a relation property in the vault · cap = the limit the Notion API enforces on that type · every name in the two middle columns was read from src/notion/schema.ts at commit 95c3558.

MEMORY · Memories · One thing an agent or a person wrote down. Forty-one properties, ten kinds. 39 columns in the reimplementation: 3 relations, 0 derived, 7 system-managed, 23 declared but left null by the vault generator.

Property	Lore name	Notion type	Flags	Note
<u>memory_id</u>	—	page id	system-managed	
title	Title	title cap 2000	title	

Property	Lore name	Notion type	Flags	Note
kind	Kind	select 10 options	discriminator	
source	Source	select 6 options	—	
status	Status	select 6 options	—	
task_state	Task State	select 5 options	task only	
blocked_by	Blocked By	rich_text cap 2 000	not modelled	A pointer to another memory, stored as free text rather than a relation.
entity_text	Entity	rich_text cap 2 000	not modelled	An entity reference stored as free text beside the Entities database.
author	Author	rich_text cap 2 000	—	A display name from LORE_USER_NAME. Not a foreign key to a user.
agent	Agent	rich_text cap 2 000	—	Free text from LORE_AGENT_NAME.
session	Session	rich_text cap 2 000	—	
topic_key	Topic Key	rich_text cap 2 000	—	The upsert key. Kebab-case path, validated on write, not unique in the store.
revision_count	Revision Count	number	system-managed	Incremented on upsert. The revisions themselves are page blocks.
keywords	Keywords	rich_text cap 2 000	not modelled	
synopsis	Synopsis	rich_text cap 2 000	not modelled	Soft-capped at 500 chars so listings need no body fetch.
compare_notes	Compare Notes	rich_text cap 2 000	not modelled	An append-only NDJSON audit log inside one 2 000-character cell. Appending past the cap throws.
promotion_source_key	Promotion Source Key	rich_text cap 2 000	system-managed not modelled	
review_by	Review By	date	not modelled	

Property	Lore name	Notion type	Flags	Note
done_at	Done At	date	task only not modelled	
decided_at	Decided At	date	decision only not modelled	
last_referenced_at	Last Referenced At	date	system-managed not modelled	Read-citation time. Distinct from last_edited_time, which tracks writes.
alternatives	Alternatives	rich_text cap 2 000	decision only not modelled	
consequences	Consequences	rich_text cap 2 000	decision only not modelled	
scope_kind	Scope Kind	select 9 options	not modelled	
scope_key	Scope Key	rich_text cap 2 000	not modelled	
audience	Audience	rich_text cap 2 000	not modelled	
lifetime	Lifetime	select 5 options	not modelled	
expires_at	Expires At	date	not modelled	
expires_on	Expires On	rich_text cap 2 000	not modelled	
pinned	Pinned	checkbox	not modelled	
pinned_priority	Pinned Priority	number	not modelled	
mutability	Mutability	select 2 options	not modelled	
supersedes_id → memory.memory_id	Supersedes	relation cap 100	self-relation	A self-relation. single_property, so the reverse edge is not maintained.
affects_id → memory.memory_id	Affects	relation cap 100	self-relation not modelled	

Property	Lore name	Notion type	Flags	Note
<u>compared_with_id</u> → memory.memory_id	Compared With	relation cap 100	self-relation not modelled	Symmetry is written by the caller, not enforced by the store.
body	page blocks	body	—	The text. Property filters cannot see it.
created_time	—	timestamp	system-managed	
last_edited_time	—	timestamp	system-managed	Overwritten in place. There is no edit history to query.
archived	—	system	system-managed	In the trash. Queries skip it; relations pointing at it do not.

Underlined = primary key · → = foreign key in the reimplementation, a relation property in the vault · cap = the limit the Notion API enforces on that type · every name in the two middle columns was read from src/notion/schema.ts at commit 95c3558.

ENTITY · Entities · The canonical handle for a thing. Added after the first four databases shipped. 7 columns in the reimplementation: 2 relations, 0 derived, 1 system-managed, 0 declared but left null by the vault generator.

Property	Lore name	Notion type	Flags	Note
<u>entity_id</u>	—	page id	system-managed	
name	Name	title cap 2 000	title	
aliases_raw	Aliases	rich_text cap 2 000	repeating group	A `·`-joined list in one cell. Lore's comment: a multi_select "would force a dataSources.update round-trip" per new alias.
kind	Kind	select 8 options	—	
description	Description	rich_text cap 2 000	—	
<u>project_id</u> → project.project_id	Project	relation cap 100	—	
<u>source_memory_id</u> → memory.memory_id	Source	relation cap 100	—	

Underlined = primary key · → = foreign key in the reimplementation, a relation property in the vault · cap = the limit the Notion API enforces on that type · every name in the two middle columns was read from src/notion/schema.ts at commit 95c3558.

FACT · Facts · A subject-predicate-object triple with two time axes and two confidences. 24 columns in the reimplementation: 5 relations, 3 derived, 6 system-managed, 6 declared but left null by the vault generator.

Property	Lore name	Notion type	Flags	Note
<u>fact_id</u>	—	page id	system-managed	

Property	Lore name	Notion type	Flags	Note
subject	Subject	title cap 2 000	title	A string. Determined by subject_entity_id, and not rewritten by a merge.
predicate	Predicate	select	—	
object	Object	rich_text cap 2 000	—	
valid_from	Valid From	date	valid time	
valid_until	Valid Until	date	valid time	Null means "still believed".
observed_at	Observed At	date	system-managed transaction time	
invalidated_at	Invalidated At	date	system-managed transaction time	
invalidated_by → memory.memory_id	Invalidated By	relation cap 100	—	The memory that prompted the retraction. Distinct from Source.
confidence	Confidence	select 3 options	—	The agent-writable stance.
confidence_score	Confidence Score	number	derived system-managed	Seeded from the stance, bumped on citation, halved on contradiction, decayed by neglect. Null until first touch.
last_referenced_at	Last Referenced At	date	system-managed not modelled	
review_by	Review By	date	not modelled	
dedup_key	DedupKey	rich_text cap 2 000	derived	A hash of normalised subject, predicate and object. An expression index, materialised.
subject_key	SubjectKey	rich_text cap 2 000	derived	Lowercased, whitespace-collapsed subject. Separate from DedupKey because 'contains' cannot match a hash.
project_id → project.project_id	Project	relation cap 100	—	
source_memory_id → memory.memory_id	Source	relation cap 100	—	Provenance: the memory that supported the claim at creation.
subject_entity_id → entity.entity_id	SubjectEntity	relation cap 100	—	
object_entity_id → entity.entity_id	ObjectEntity	relation cap 100	—	
scope_kind	Scope Kind	select	not modelled	

Property	Lore name	Notion type	Flags	Note
<u>scope_key</u>	Scope Key	rich_text cap 2 000	not modelled	
<u>lifetime</u>	Lifetime	select	not modelled	
<u>expires_at</u>	Expires At	date	not modelled	
<u>created_time</u>	–	timestamp	system-managed	

Underlined = primary key · → = foreign key in the reimplementation, a relation property in the vault · cap = the limit the Notion API enforces on that type · every name in the two middle columns was read from `src/notion/schema.ts` at commit 95c3558.

5. Normalisation

The usual normalisation exercise walks a badly-shaped relation to Boyce-Codd form and declares victory. That is not the interesting exercise here, because Lore's designers are not naïve — most of the schema is in good shape. The interesting exercise is the other one: *which violations ship, and why*. Three do, and each has a substrate reason we can name.

Figure 3 — The normalisation walk

Normalisation ladder

6 rungs · 5 still shipping in 1.0.0

UNF

already in UNF

■ SHIPPING

```
entity(entity_id, Name, Aliases, Kind, Description, Project[], Source)
```

A relation is in 1NF when every attribute holds a single atomic value.

violates Aliases is a `,`-joined list inside one rich_text cell.

ANOMALY IT LEAVES BEHIND

No predicate can ask "which entity owns the alias SVC-4470". A `contains` filter matches substrings, so it also returns entities whose alias merely contains that text, and the caller has to re-split the cell and check exactly. Two entities can hold the same alias and nothing notices.

→ Project the list into ENTITY_ALIAS(entity_id, alias) with a unique index on alias.

STILL SHIPPING BECAUSE

Deliberate, and documented. A multi_select would need a dataSources.update round trip per new alias, which for free-form aliases is a schema migration per fact.

read in src/core/entity.ts – "Aliases stored on a single rich_text cell as `,`-joined."

UNF

already in UNF

■ SHIPPING

```
memory(memory_id, ..., Tags, Compare Notes, page blocks)
```

Same rule, three more times.

violates Three more repeating groups: Tags as a multi_select, Compare Notes as append-only NDJSON in one cell, and the revision chain as page blocks.

ANOMALY IT LEAVES BEHIND

Compare Notes is an audit table inside a 2 000-character cell; Lore's own comment records that "append-past-cap throws". The revision chain is prose, so "what did this runbook say in March" is not a query anyone can write.

→ MEMORY_TAG, MEMORY_COMPARISON and MEMORY_REVISION, one relation each.

STILL SHIPPING BECAUSE

Structural. A page body is not a column, and Notion offers no child table.

1NF

already in 1NF

■ SHIPPING

```
fact(fact_id, Subject, Predicate, Object, SubjectKey, DedupKey, ...)
```

A derived attribute should be a view, not a stored column, unless the store cannot index the expression. violates **SubjectKey** and **DedupKey** are stored derivations of the other columns.

ANOMALY IT LEAVES BEHIND

The store cannot. There are no expression indexes, so the expression is materialised, can drift from its source, and needs a backfill migration ('lore migrate --dedup-keys') for every row written before it existed.

→ CREATE UNIQUE INDEX ON fact (lower(subject), predicate, lower(object)). One line, no column, no migration.

STILL SHIPPING BECAUSE

A faithful workaround for a real limitation, and the clearest single example of the substrate showing through the schema.

3NF

already in 3NF

■ SHIPPING

```
fact(fact_id, Subject, Predicate, Object, SubjectEntity, ...)
```

A relation is in BCNF when every determinant is a candidate key. violates **SubjectEntity** → **Subject**. The determinant is not a candidate key.

ANOMALY IT LEAVES BEHIND

Subject is a title string and **SubjectEntity** is a relation to the canonical row; the relation determines the string. 'mergeEntities' repoints every fact relation from loser to winner and never rewrites the title, so after a merge the vault holds facts whose title says one name and whose relation points at another. §7.2 counts them.

→ Drop the **Subject** string; derive it from the relation at read time.

STILL SHIPPING BECAUSE

Structural. Every Notion database needs a title property, and a title property cannot be a relation.

read in src/core/entity-merge.ts – the merge moves fact relations and aliases; the word **Subject** does not appear in it.

BCNF

already in BCNF

■ SHIPPING

fact(..., Confidence Score)

A stored derivation is only safe when its inputs are stored and the derivation is recomputable.

violates Confidence Score is a stored derivation of citation, contradiction and decay events that are not themselves stored.

ANOMALY IT LEAVES BEHIND

The score is a state machine over an event stream, so two vaults that saw the same evidence in a different order hold different scores for the same fact. Nothing can recompute or audit it. Lore ships `lore migrate --audit-fact-confidence`, which reports the distribution rather than the correctness.

→ Store the citations, contradictions and touches; make the score a view over them.

STILL SHIPPING BECAUSE

The score is then excluded from ranking anyway — `confidenceFactor` returns 1.0 for every value.

target

the schema the reimplementation runs

terminal form

The schema the SQLite and PostgreSQL arms execute.

Every repeating group is a relation; every derivation is an index or a view; every determinant is a candidate key.

No anomaly is left to remove: every repeating group is a relation, every derivation is an index or a view, and every determinant is a candidate key.

This is what the reimplementation runs.

Each rung names the form the relation is *already* in, the rule the next form imposes, the anomaly that survives until you climb, and the one-line fix. The tag marks a rung whose violation is present in the released product; where Lore explains why, the explanation and the file it was read from are printed beneath. Read the tagged rungs alone and you have the section: an agent memory store on this substrate does not merely denormalise for speed, it denormalises because the store has no other shape to offer.

Every violation on this ladder was read from Lore's own schema at commit 95c3558, not invented for the exercise, and 5 of the 6 rungs are tagged SHIPPING because they are still there. That is the finding of Section 5: these are not oversights waiting on a patch, they are what the substrate costs, and the last rung is the only place the anomalies actually stop.

Rungs marked as shipping are violations still present in Lore. They are the argument of this section; the rest is the ladder that gets us to them.

5.1 A repeating group in one cell

`Entities.Aliases` is a single `rich_text` cell holding a comma-joined list. This is a first normal form violation of the most textbook kind, and Lore states its reason plainly:

"Multi-select option lists require a schema migration whenever a new alias appears, and aliases are deeply free-form (case variants, 'MemoryService.create' alongside 'MemoryService', legacy spellings) — every new fact would force a dataSources.update round-trip."

src/notion/schema.ts

The reasoning is sound and the consequence is real. Because the list lives inside a string, the only server-side operator that reaches it is `contains` — a substring test over the whole cell. Lore does the correct thing: it uses the substring filter as a *candidate* filter and then re-splits every candidate in the client to keep exact tokens. Correctness survives. What does not survive is the claim that the store answered the question, and the invariant "an alias identifies one entity" is not merely unenforced but explicitly disclaimed — the source comment reads "aliases are deliberately not unique across entities". \$9.5 measures what the over-matching costs, and the honest answer is: less than we expected.

The same violation appears three more times on Memories, and one of the three is worse than the alias case. Compare `Notes` is an append-only NDJSON audit log — one JSON line per adjudication verdict — stored in a single `rich_text` cell, and Lore's comment records that appending past the 2 000-character cap throws. That is a child table inside an attribute, with a hard row limit, and the revision chain of a topic-keyed memory is worse still: it is prose in page blocks, so "what did this runbook say in March" is not a question anyone can write a query for.

5.2 An index, materialised as a column

`Facts` carries two derived columns. `DedupKey` is a hash of normalised subject, predicate and object, used to coalesce cosmetic duplicates. `SubjectKey` is the lowercased, whitespace-collapsed subject. A relational engine expresses both as one line — `CREATE UNIQUE INDEX ON fact (lower(subject), predicate, lower(object))` — because it has expression indexes. Notion does not, so the expression becomes a stored column, which can drift from its source, and which needs a backfill migration for every row written before it existed.

Lore's comment even explains why *two* columns are needed rather than one, and the reason is a second substrate limitation stacked on the first: they need `contains` substring matching, "which Notion doesn't run against hashed values". A hash serves uniqueness; a normalised string serves lookup; a database would have served both with one index and no columns.

5.3 The violation in the middle of the graph

Every Notion database has exactly one title property, and a title property cannot be a relation. `Facts` therefore carries its subject twice: as `Subject`, a string, and as `SubjectEntity`, a relation to the canonical `Entity` row. The relation determines the string. Neither is a candidate key of `Facts`. That is a Boyce-Codd violation, and it is not hypothetical — it is reachable by the most ordinary maintenance operation the system has.

`mergeEntities` repoints every fact relation from the losing entity to the winner, absorbs the loser's lookup forms as aliases, and archives the loser. It never rewrites the `Subject` title. After a merge, the vault holds facts whose title says one name and whose relation points at a row with another. In our generated vault, 105 facts are in that state — 2.5% of all facts — and an agent reading the title reads the old name while an agent following the relation reads the new one.

Table 4 — Functional dependencies

Relation	Dependency	Status
memory	memory_id → title, kind, source, status, author, agent, session, topic_key, revision_count, body	holds
memory	topic_key, {project_id} → memory_id	unenforced Lore's upsert key. Its own comment: "Notion provides no per-key uniqueness enforcement." §6.3 breaks it.
fact	fact_id → subject, predicate, object, valid_from, valid_until, observed_at, invalidated_at, confidence, confidence_score	holds
fact	subject_entity_id → subject	BCNF The determinant is not a candidate key of FACT.
fact	subject, predicate, object → dedup_key, subject_key	stored derivation An expression index written as columns.
fact	dedup_key → fact_id	unenforced Coalescing key. Nothing makes it unique.
entity	entity_id → name, aliases_raw, kind, description	holds
entity_alias	alias → entity_id	does not hold Lore is explicit: "aliases are deliberately not unique across entities."
project	project_id → name, type, path, status	holds

Table 5 — The normalisation walk, tabulated

From	Violation	Anomaly it permits	Ships?
UNF	Aliases is a ``, ``-joined list inside one rich_text cell.	No predicate can ask "which entity owns the alias SVC-4470". A `contains` filter matches substrings, so it also returns entities whose alias merely contains that text, and the caller has to re-split the cell and check exactly. Two entities can hold the same alias and nothing notices.	ships
UNF	Three more repeating groups: Tags as a multi_select, Compare Notes as append-only NDJSON in one cell, and the revision chain as page blocks.	Compare Notes is an audit table inside a 2 000-character cell; Lore's own comment records that "append-past-cap throws". The revision chain is prose, so "what did this runbook say in March" is not a query anyone can write.	ships
1NF	SubjectKey and DedupKey are stored derivations of the other columns.	The store cannot. There are no expression indexes, so the expression is materialised, can drift from its source, and needs a backfill migration ('lore migrate --dedup-keys') for every row written before it existed.	ships
3NF	SubjectEntity → Subject. The determinant is not a candidate key.	Subject is a title string and SubjectEntity is a relation to the canonical row; the relation determines the string. `mergeEntities` repoints every fact relation from loser to winner and never rewrites the title, so after a merge the vault holds facts whose title says one name and whose relation points at another. \$7.2 counts them.	ships
BCNF	Confidence Score is a stored derivation of citation, contradiction and decay events that are not themselves stored.	The score is a state machine over an event stream, so two vaults that saw the same evidence in a different order hold different scores for the same fact. Nothing can recompute or audit it. Lore ships 'lore migrate --audit-fact-confidence', which reports the distribution rather than the correctness.	ships

The 'ships' column is the finding. Everything above it is the standard exercise.

A fourth item belongs here even though it is not a normal-form violation in the classical sense. `Confidence Score` is a stored derivation whose inputs are not stored. Lore seeds it from the categorical stance (certain → 0.9, likely → 0.6, speculative → 0.3), raises it by 0.05 of the remaining headroom on each read citation, multiplies it by 0.5 on a contradiction, and decays it at 0.99 per day of neglect after a 60-day grace period. Because the citations, contradictions and touches are not themselves rows, the score cannot be recomputed, audited or rolled back, and two vaults that saw the same evidence in a different order hold different scores for the same fact.

The most striking thing about the score is what happens to it next. Lore maintains it, pays write amplification to persist it, and then declines to let it influence retrieval at all — `confidenceFactor` returns 1 for every input, with the comment that "ranking callers cannot use confidence as a multiplier". It is a trust label, not a probability, and reading it as either a probabilistic database tuple weight^[58] or an AGM belief state^[57] would misrepresent it. We think declining to rank on it is the right call — a citation is a retrieval event, not a corroboration, so a frequently-retrieved wrong fact would otherwise ratchet upward — but it leaves a maintained column that nothing consumes.

6. The temporal model

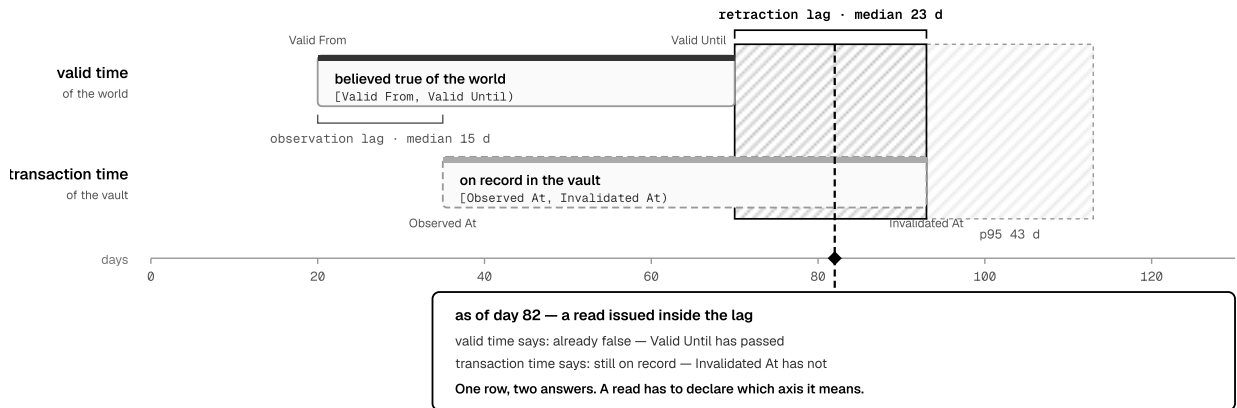
Lore's Facts relation is bitemporal in schema, which puts it ahead of most agent memory systems and level with Zep's Graphiti^{[6][7]}. Valid time says when a claim was true; transaction time says when the vault believed it. The standard has had the vocabulary since SQL:2011^[44] and the consensus glossary since 1998^[43].

The question a review has to ask is whether the second axis pays for itself, and the honest way to answer it is to count the rows where the two axes disagree. If they never disagree, the schema is carrying a column for nothing. In our vault they disagree often: retractions lag the world by a median of 23 days and a 95th percentile of 43, and at four probe instants between 1.3% and 5.7% of all facts answer differently depending on which axis the question asked about.

Figure 4 — Two axes for one fact

Two time axes, one fact

retraction lag · median 23 d · p95 43 d



Both bands are half-open: $[start, end)$. The days the bands begin and end are schematic; every hatched distance is measured — the observation lag is the median 15 days between a claim becoming true and being written down, and the retraction lag is the median 23 days between it ceasing to be true and being invalidated, extended to the 43-day 95th percentile. Highlighted: the window where a read gets a different answer depending on which axis it was written against. Lore stores both; a query has to mean one.

Probed against the whole vault, the two axes disagree on 245 as of day 120 (5.7%), 123 as of day 240 (2.9%), 112 as of day 360 (2.6%), 55 as of day 480 (1.3%) — out of 4 277 facts carrying a validity interval, of which 1 196 have been closed and 1 172 of those (98.0%) were retracted only after the claim had already stopped being true. The two axes disagree often enough that choosing between them is a correctness decision, not a formality.

Valid time above, transaction time below, the lag between them shaded. An audit that asks the wrong axis gets a defensible-looking wrong answer.

Table 6 — Where the axes disagree

A row disagrees on day D when “was this true on D ” and “did the vault believe it on D ” give different answers. Choosing the wrong axis is a correctness error, not a formality.

Probe day	Rows where the axes disagree	Share of all facts
120	245	5.7%
240	123	2.9%
360	112	2.6%
480	55	1.3%
Retraction lag behind reality, median / 95th percentile		23 / 43 days

Two caveats, both of which cut against a simple reading. First, Notion's own `last_edited_time` is not transaction time in Snodgrass's sense: it is overwritten in place, so there is no history to query and no way to reconstruct what the vault believed before the last edit. Lore's `Observed At` and `Invalidated At` are doing the work the substrate does not do, and they can do it only once per fact — one observation and one retraction, not a history of belief. Second, our reading of Lore's query paths

suggests the surface is narrower than the schema: the `asOf` parameter applies a single date, and we did not find a path that filters valid time and transaction time independently. A bitemporal schema with a single-axis query surface is a common and recoverable state — the columns are there when someone wants them — but it is worth naming, and we flag it as inference rather than measurement in §14.

7. Methodology

7.1 The vault, generated backwards

Relevance labels chosen by looking at what a store returned are worthless, because the store gets to define what counts as right. So nothing in our corpus is labelled. A world is generated first — services, people, teams, incidents, decisions — and the facts are read off that world by construction, with ownership histories that open and close over 540 simulated days. Memories are then *rendered* from facts: several natural-language restatements of the same triple, scattered across sessions and authors. The correct answer to “what is currently true about payments-api” is therefore definitional: it is the set of facts the generator emitted with an open interval for that subject.

Table 7 — The generated vault

Seed 20260822, scale 5. Regenerating with the same seed reproduces this vault exactly.

Relation	Rows	Composition
Memories	11 885	112 archived
Facts	4 277	3 081 open, 1 196 closed
Entities	1 096	1 153 aliases in 1 096 cells
Projects and topics	35	6 projects
Sessions	1 600	across 540 simulated days
Questions	361	10 classes

A clean vault would prove nothing, so six defects are injected deliberately. Each is a state the substrate cannot refuse, and each is recorded with its ground-truth extent so that §9 reports detection rates rather than impressions.

Table 8 — Defects injected on purpose

Defect	Count	Why it is there
Duplicate entity rows	40	the same service introduced twice, by two sessions
Facts with a stale Subject title	105	the state an entity merge leaves behind
Subject-predicate pairs with overlapping validity	50	a new owner asserted without closing the old one
Memories archived after facts cite them	112	a relation pointing at a page in the trash
Aliases that are a prefix of another	57	so a substring filter over the joined cell over-matches
Distractor memories	1 849	name an identifier, assert nothing about it

7.2 The workload

The 10 question classes are read off Lore's own surface, not chosen to make a point. If `lore ask <entity>` exists, the workload contains an `ask-entity` question; if `lore conflicts scan` exists, it contains a `conflict scan`. Each class is annotated with the minimum relational algebra it needs, and with what the documented Data API can state as a query.

Table 9 — The question classes

Class	Raised by	Minimum algebra	Instances	Data API
Wake-up context	lore hooks wakeup	$\sigma \pi$ sort limit	40	expressible
Everything about an entity	lore ask <entity>	$\sigma \cup \bowtie$	40	partial
Where did this claim come from	lore-fact action=query	$\sigma \bowtie$	40	client-side join
What did we believe on date D	temporal recall	σ range	40	expressible
What is true now	lore-context action=wake-up	σ	40	expressible
Counting and grouping	lore status, digest synthesis	$\sigma \gamma$	40	not expressible
Find contradictions	lore conflicts scan	$\sigma \bowtie (\text{self}) \gamma$	1	not expressible
Follow a decision chain	lore-decision action=supersede	σ transitive closure	40	not expressible
Search what a memory says	lore search <query>	σ (full text)	40	not expressible
Two joins deep	digest synthesis	$\sigma \bowtie \bowtie$	40	client-side join

Of the 361 question instances, 44.6% need a join, 11.4% need an aggregate, and 11.1% need a transitive closure. Together, 66.8% of the workload needs something the Data API cannot state as a filter.

7.3 The arms

Three stores answer every question. The **Notion arm** is an emulator whose surface is restricted to operations the public reference documents, and whose refusals are restricted to things that reference does not offer. The **SQLite arm** runs the reimplemented schema in process through `node:sqlite`, with secondary indexes, foreign keys and an FTS5 index over bodies. The **PostgreSQL arm** runs the same schema through PGlite, adding range-typed validity, a GiST exclusion constraint, and a GIN index over `to_tsvector`. Two ablations drop every secondary index.

What the emulator is not. We hold no Notion workspace and issue no requests. Wall-clock through an emulator measures our own JavaScript, so we do not report it. What we report instead are counts — requests, bytes, and rows the client had to examine — which are fixed by the API's shape rather than by anyone's network. The documented average of 3 requests per second then converts a request count into a floor on wall-clock that no bandwidth removes. §14 states what this cannot capture.

7.4 Scoring

Answers here are sets, not ranked lists, because every question in this workload has a definite answer rather than a best guess. We report set F_1 and, more importantly, the *exact* rate — the share of questions where the returned set is the correct set with nothing added and nothing missing. For a memory system that is the number that matters: an agent acting on a nearly-correct set of facts acts wrongly.

8. Expressibility and cost

The first result is the one we did not expect to have to lead with: **the vault is correct**. Across the whole workload the Notion arm returns the exactly-correct set for 100.0% of questions, the same as SQLite and within a rounding of PostgreSQL, whose 3 divergences are the full-text stemming case dissected in §10.2 rather than an error.

A critique that expected the substrate to give wrong answers would stop here, and it would be wrong to. The Data API is not a bad store. It is a store that will do the work if you send it enough requests — and the question this paper is actually about is how many.

Table 10 — Correctness and cost, by arm

Arm	Exact	F_1	Requests	Bytes in	Rows scanned client-side
Lore on Notion <i>emulated</i>	100.0%	1.000	1 447.9	658.7 kB	1 519.6
SQLite 3.x	100.0%	1.000	1.0	—	0.0
PostgreSQL	99.2%	0.992	1.0	—	0.0
SQLite, no indexes	100.0%	1.000	1.0	—	0.0
PostgreSQL, no indexes	99.2%	0.992	1.0	—	0.0

Table 11 — Exact-answer rate by class

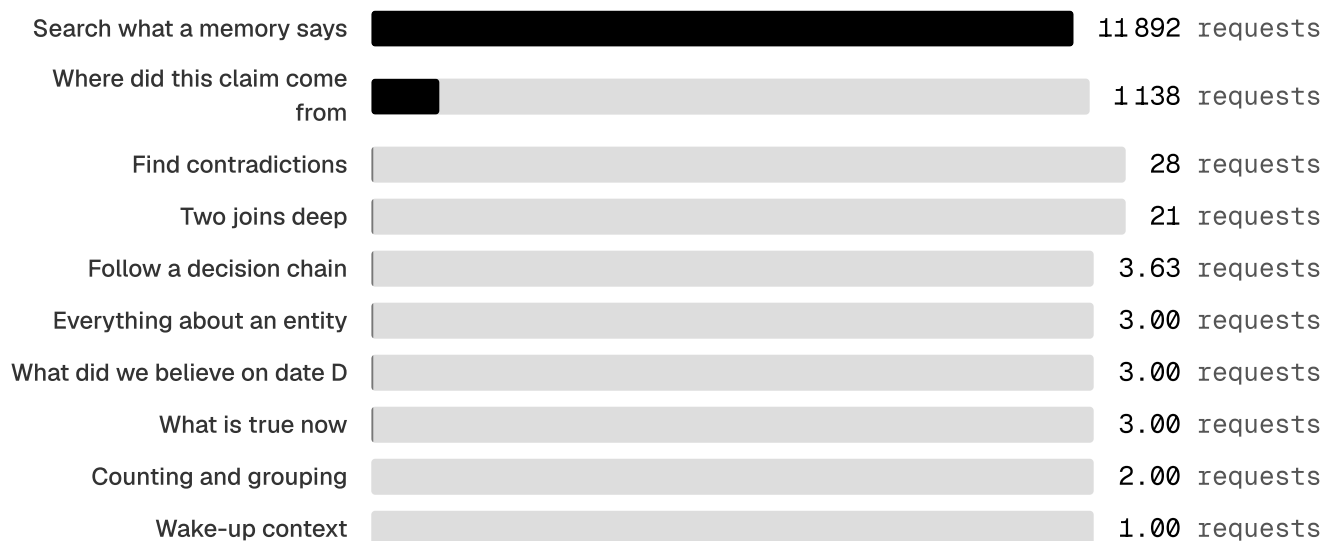
Class	Lore on Notion	SQLite 3.x	PostgreSQL
Wake-up context	100%	100%	100%
Everything about an entity	100%	100%	100%
Where did this claim come from	100%	100%	100%
What did we believe on date D	100%	100%	100%
What is true now	100%	100%	100%
Counting and grouping	100%	100%	100%
Find contradictions	100%	100%	100%
Follow a decision chain	100%	100%	100%
Search what a memory says	100%	100%	93%
Two joins deep	100%	100%	100%

Flat, and that is the point. The differences between these stores are not in this table.

8.1 Amplification

Every question in this workload is one statement in SQL. The same questions cost the Data API a mean of 1 447.9 requests, and the distribution is wildly uneven: two classes cost two or three requests, and two cost more than a thousand.

Figure 5 — Requests per question, by class



Logarithmic, because a linear axis would render eight of the ten classes as a flat line against body search. Every bar is a mean over that class's instances.

Table 12 — Request amplification

Requests are means over the instances of each class. The floor is what Notion's documented 3 requests per second implies for one token; it is a lower bound on wall-clock, not a measurement of it.

Class	Notion requests	SQL statements	Amplification	Rows in client	Rate-limit floor
Search what a memory says	11 892	1	11 892×	11 773	66.1 min
Where did this claim come from	1 138	1	1 138×	1 126	6.3 min
Find contradictions	28	1	28.0×	4 630	9.3 s
Two joins deep	21	1	21.1×	539	7.0 s
Follow a decision chain	3.63	1	3.6×	0	1.2 s
Everything about an entity	3.00	1	3.0×	0	1.0 s
What did we believe on date D	3.00	1	3.0×	0	1.0 s
What is true now	3.00	1	3.0×	0	1.0 s
Counting and grouping	2.00	1	2.0×	160	670 ms
Wake-up context	1.00	1	1.0×	0	330 ms

8.2 The join that is not there

Provenance is the cleanest case, and it is one question: *which claims about this predicate came from a memory written by this author?* The author lives on Memories. The fact points at the memory through a Source relation. In SQL that is a join and the planner picks an index. Through the Data API there is no way to filter Facts by a property of the page its relation points at, so the client queries Facts, then retrieves each candidate memory by identifier, then filters on author locally: 1 138 requests, 1 126 rows examined in the client, 6.3 min of floor.

Figure 6 — One join, two shapes

One join, two shapes

1138 requests vs 1 statement · 1138×

Notion Data API

lore-fact action=query · a client-side join

three phases

1138 requests

POST /v1/databases/{Facts}/query

× 12

100 rows per page. The API offers no join, so the query can only return the fact rows.

GET /v1/pages/{fact.Source}

× 1126

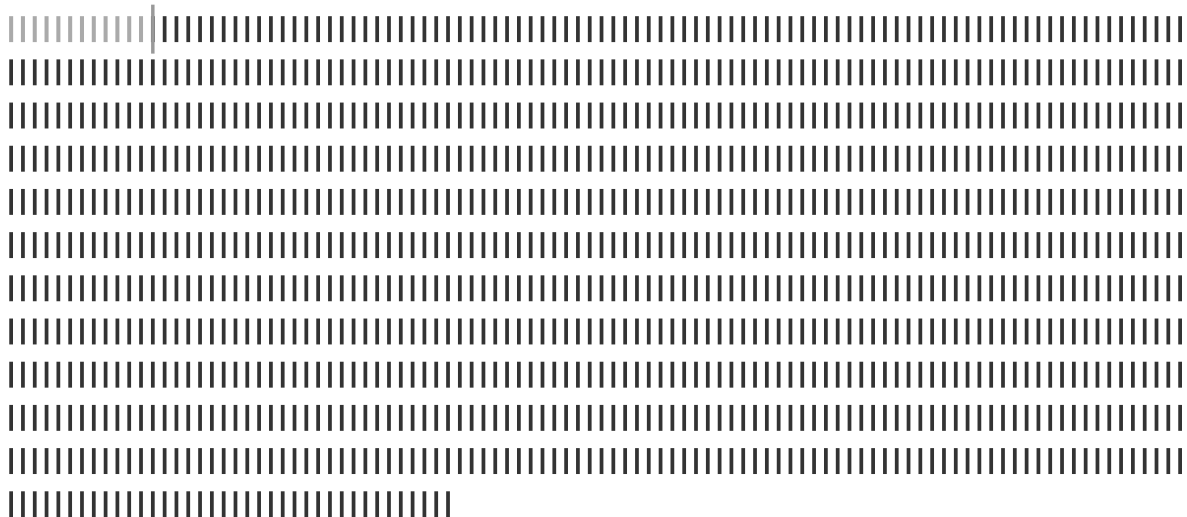
One retrieval per fact. Author lives on the memory page, on the far side of a relation.

filter on Author

in the client

1126.2 rows the caller holds in its own memory and scans itself.

12 paged query requests, then 1126 page reads – one per fact



one tick = one request · one row = 100

1138 requests

floor at 3 req/s 6.3 min · 835.8 kB received

SQLite · PostgreSQL

one statement · the engine owns the join

one statement

1 round trip

```
SELECT f.subject, f.predicate, f.object,
       m.title, m.author
FROM   fact f
JOIN   memory m ON m.memory_id = f.source_memory_id
WHERE  m.author = ?;
```

the join happens inside the engine – nothing crosses a network

| 1 statement · 1 round trip

same field, same tick pitch, same scale

1 request

floor at the same 3 req/s 333 ms · 0 rows joined in the client

Each tick is one HTTP request; each row of ticks is 100. Both panels are drawn into the same box at the same tick pitch, so the single accented mark on the right is exactly one request wide. Counts are the mean over the provenance questions in the workload, rounded to whole requests for the drawing; the unrounded means are in the table behind this figure. The SQL floor is the same 3 requests per second applied to one statement — no rate limit applies to a local engine, and it is shown at Notion's ceiling only so the two numbers share a scale.

Provenance is the cheapest join a memory store can be asked for: a fact, the memory it came from, and who wrote it. The Notion arm answers it correctly and pays 1 138 HTTP round trips for the mean question, 1 126 of them a single page retrieval issued because a relation cannot be followed inside a query. At the documented 3 requests per second that is a floor of 6.3 min before any row is joined. The same question is one statement, and the engine does the join.

The comb on the left is one request per candidate fact. The bar on the right is one statement. Both return the same set.

The other join-shaped classes behave the same way at smaller scale. Following a decision chain costs 3.63 requests because Supersedes is a `single_property` self-relation with no reverse edge, so “what replaced this” is a filter and the walk pays a request per hop; a recursive CTE does the whole closure in one statement. Two joins deep — facts about entities introduced by memories a given author wrote — costs 21.1, and the client has to chunk its own disjunction because `relation` contains takes one page identifier at a time and the payload cap is 500.0 kB.

Finding contradictions is the aggregate case. It is a self-join with an overlap predicate, and there is no way to express either, so the client fetches every fact on a functional predicate and compares them pairwise: 28 requests and 4 630 row-comparisons

here. This is why Lore ships `lore_conflicts_scan` with a documented cap of 500 raw candidates per project, and why bypassing the cap is described in its own source as “full $O(n^2)$ coverage”.

9. Integrity

Cost is the half of the story a reader expects. The other half is what the store will refuse, and it is the half where the gap is not a multiple but a categorical difference. A database's central service is not answering questions; it is declining to enter states that would make future answers wrong.

We took 7 invariants from the schema, named the DDL that would enforce each one declaratively, and attacked them. 4 of the 7 have no declarative enforcement anywhere in the substrate.

Table 13 — Invariants and what enforces them

ID	Invariant	Declarative form	Enforced in the vault by	Broken by
INV-1	At most one memory per (topic_key, project set).	<code>UNIQUE (topic_key, project_id)</code>	application read-then-write, in memory-topic-key.ts	concurrent upsert
INV-2	Every fact's Source references a memory that exists.	<code>FOREIGN KEY (source_memory_id) REFERENCES memory ON DELETE RESTRICT</code>	nothing	archiving a memory page
INV-3	A subject holds at most one object per functional predicate at any instant.	<code>EXCLUDE USING gist (subject_entity_id WITH =, predicate WITH =, validity WITH &&)</code>	nothing	two agents asserting different owners without closing the first interval
INV-4	confidence $\in$ {certain, likely, speculative}.	<code>CHECK (confidence IN (...))</code>	Notion select options, which an editor may extend by hand in the UI	a human typing a new option into the property
INV-5	An alias identifies exactly one entity.	<code>UNIQUE (alias)</code>	nothing	two entities independently claiming the same alias
INV-6	<code>valid_from ≤ valid_until</code> .	<code>CHECK (valid_until IS NULL OR valid_from ≤ valid_until)</code>	not verifiable from the source we read	a backdated correction
INV-7	<code>dedup_key = hash(normalise(subject), predicate, normalise(object))</code> .	<code>GENERATED ALWAYS AS (...) STORED, or a plain expression index</code>	FactService.create at write time, plus a backfill migration for older rows	a write that bypasses the service — a hand edit in the Notion UI

Figure 7 — What the constraint would have been

What would enforce it, what does, what breaks

7 invariants · 4 with nothing declarative behind them

INV-1 At most one memory per (topic_key, project set).

WOULD ENFORCE IT

UNIQUE (topic_key, project_id)

→ ENFORCES IT IN LORE

application read-then-write, in memory-topic-key.ts

→ WHAT BREAKS

concurrent upsert

Two parallel saves with the same topicKey can both find no existing match and both create fresh rows.

INV-2 Every fact's Source references a memory that exists.

NOTHING ENFORCES THIS

WOULD ENFORCE IT

FOREIGN KEY (source_memory_id) REFERENCES memory ON DELETE RESTRICT

-X ENFORCES IT IN LORE

nothing; `lore debt scan` finds the orphans afterwards

→ WHAT BREAKS

archiving a memory page

INV-3 A subject holds at most one object per functional predicate at any instant.

NOTHING ENFORCES THIS

WOULD ENFORCE IT

EXCLUDE USING gist (subject_entity_id WITH =, predicate WITH =, validity WITH &&)

-X ENFORCES IT IN LORE

nothing; `lore conflicts scan` finds the contradictions afterwards, capped at 500 candidates per project

→ WHAT BREAKS

two agents asserting different owners without closing the first interval

INV-4 confidence $\in$ {certain, likely, speculative}.

WOULD ENFORCE IT

CHECK (confidence IN (...))

→ ENFORCES IT IN LORE

Notion select options, which an editor may extend by hand in the UI

→ WHAT BREAKS

a human typing a new option into the property

INV-5 An alias identifies exactly one entity. NOTHING ENFORCES THIS

WOULD ENFORCE IT

UNIQUE (alias)

-X ENFORCES IT IN LORE

nothing — and Lore states the invariant does not hold

→ WHAT BREAKS

two entities independently claiming the same alias

INV-6 valid_from $\leq$ valid_until. NOT VERIFIABLE

WOULD ENFORCE IT

CHECK (valid_until IS NULL OR valid_from <= valid_until)

-X ENFORCES IT IN LORE

not verifiable from the source we read

→ WHAT BREAKS

a backdated correction

INV-7 dedup_key = hash(normalise(subject), predicate, normalise(object)).

WOULD ENFORCE IT

GENERATED ALWAYS AS (...) STORED, or a plain expression index

→ ENFORCES IT IN LORE

FactService.create at write time, plus a backfill migration for older rows

→ WHAT BREAKS

a write that bypasses the service — a hand edit in the Notion UI

The arrow between the first two lanes is the figure: unbroken where something in Lore stands in for the constraint, crossed through where nothing does. A crossed arrow does not mean the invariant is unimportant — it means the store will accept a row that violates it and no one will know until a scan is run, which is precisely the class of failure §6 measures. The declarative column is what the SQLite and PostgreSQL arms actually execute; PostgreSQL is the only one of the three engines that can express all of it.

Each of these is one line of standard DDL on any relational engine. On the Notion substrate 4 of 7 have no declarative enforcement at all — 3 of them are guarded by nothing whatsoever and are found after the fact by a scan, and the rest survive only as application-level convention that a concurrent write, an archived page or a hand edit in the Notion UI walks straight through.

For each invariant: the one line of DDL that would have declared it, what actually upholds it, and what breaks it.

9.1 Concurrent upsert

Lore upserts a topic-keyed memory by reading the vault for a matching row and then either creating or patching. The Data API documents no conditional write, so the window between the read and the write cannot be closed from the client. Lore knows this and says so:

“Concurrent upserts remain a single-agent serial workflow. Two parallel saves with the same topicKey can both find no existing match and both create fresh rows with Revision Count: 1; Notion provides no per-key uniqueness enforcement.”

src/core/memory-topic-key.ts

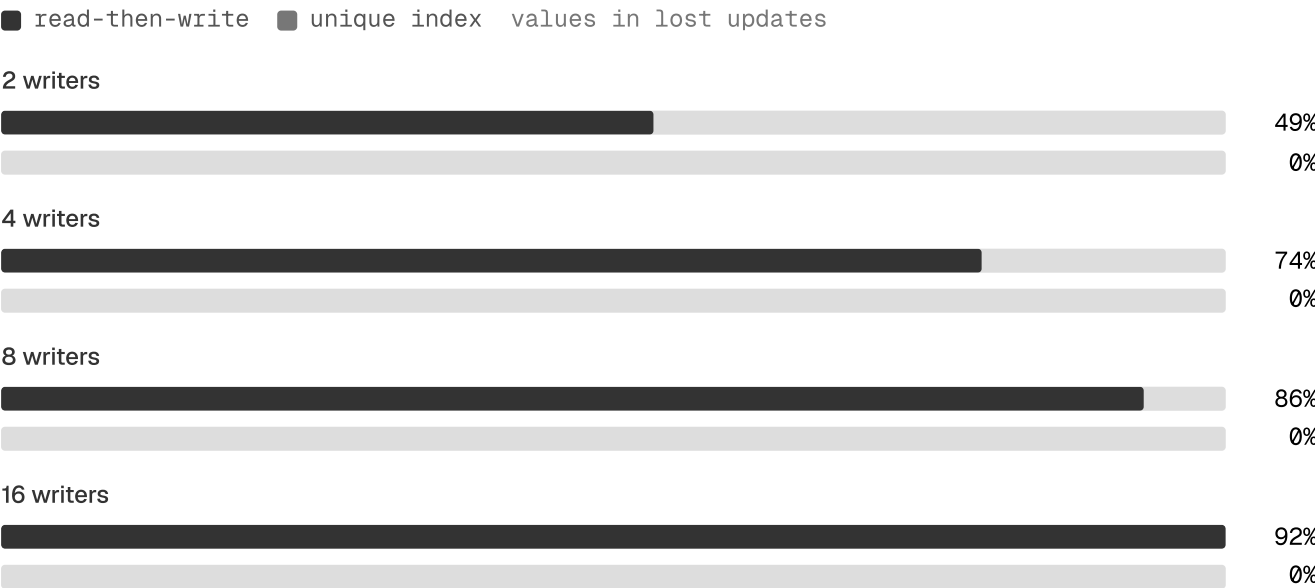
We ran the interleave explicitly rather than depending on a scheduler, because a scheduler-dependent result is not a measurement. With 8 writers over 200 rounds, every writer reading before any writer writes — the worst case the protocol admits and the one the API gives no way to exclude — the read-then-write path lost 87.1% of its 1 600 updates and left 7 duplicate rows for a key that should have one. The same workload through INSERT ... ON CONFLICT against a unique index lost 0.0% and left 0. This is the lost update of Gray^[51] and Berenson^[52], arriving in 2026 by a new route.

Table 14 — Concurrent upsert on one topic key

8 writers, 200 rounds, 1600 attempted writes. The interleave is explicit rather than scheduler-dependent: every writer reads before any writer writes, which is the worst case the protocol admits and the one the API gives no way to exclude.

Protocol	Rows for one key	Duplicate rows	Lost updates	Loss rate
Read-then-write, no conditional write	8	7	1 393	87.1%
INSERT ... ON CONFLICT on a unique index	1	0	0	0.0%

Figure 8 — Loss against writer count



Measured at 2, 4, 8 and 16 writers. The unique-index arm is flat at zero, which is why it is drawn rather than omitted.

9.2 A temporal primary key

Some predicates are functional: a service has one owner at a time. That is a constraint, and PostgreSQL can declare it — an exclusion constraint over a range type saying that no two rows may share a subject and a predicate while their validity intervals overlap. Once declared, the write that would create the contradiction is the write that fails, and no scan is needed because the state is unreachable.

The experiment has two halves, and the first half is the more telling. We built the constraint against the vault *as generated* and PostgreSQL refused to create it, because 70 subject-predicate pairs already violate it. The engine declined to certify data written without it. The second half loads the same rows one at a time with the constraint in place: 2 532 accepted, **192 rejected**, and the rejections are precisely the writes that would have created a contradiction. The vault rejected 0 of them. We want to be fair about what this shows. Lore is not unaware of contradictions — it ships a scanner for them, with a verdict workflow and an adjudication trail, and the scanner exists precisely because the store cannot refuse the write. The comparison is between *preventing* a state and *detecting* it afterwards at $O(n^2)$ cost under a cap of 500. Detection is a real and useful thing to ship. It is not the same thing.

Table 15 — The exclusion constraint

Facts on a functional predicate	2 724
Subject-predicate pairs asserting two objects at once	70
Writes the vault refused	0
Writes PostgreSQL refused under the exclusion constraint	192
Rows held after those refusals	2 532
Constraint added to the vault as written	refused <pre>Key (subject_entity_id, predicate, int8range(COALESCE(valid_from, '-999999'::integer::bigint), COALESCE(valid_until, 999999::bigint), '[]'::text))=(ent_0003an, owned_by, [177,295)) conflicts with key (subject_entity_id, predicate, int8range(COALESCE(valid_from, '-999999'::integer::bigint), COALESCE(valid_until, 999999::bigint), '[]'::text))=(ent_0003an, owned_by, [205,330)).</pre>

9.3 Referential integrity

A relation property is not a foreign key. It holds page identifiers, and nothing checks them at write time or complains at read time. When a memory that facts already cite is archived, every relation pointing at it stays intact and stays wrong. In our vault 112 such memories were archived, leaving 112 claims whose provenance the store can no longer return. Under `ON DELETE RESTRICT` the same operation is refused 112 times out of 112 and leaves 0 orphans.

This one bit us, twice, inside our own tooling. The subsampling routine that builds smaller vaults for the scaling experiment has to compute referential closure by hand over five foreign keys; both times we missed one, and both times the SQLite arm failed immediately and loudly while every other path would have carried the broken state silently. We have left the incident in a comment in the source, because it is the paper's argument arriving unannounced in the paper's own machinery.

Table 16 — Archiving a cited memory

Store	Deletes attempted	Refused	Claims left without provenance
Relation property, no foreign key	112	0	112
FOREIGN KEY ... ON DELETE RESTRICT	112	112	0

9.4 Entity resolution

Nothing gives a Notion title a unique index, so the same real thing can be introduced twice by two sessions — once under its name and once under its handle. Our vault contains 40 such pairs, 3.65% of all entities, and 39 subjects have their facts split across both rows. An agent asking about one row gets a partial answer and no signal that a second row exists.

Lore's merge is careful — non-destructive, idempotent on retry, and archiving the loser only after every fact relation has moved — but it is explicitly single-hop, so a chain $A \rightarrow B \rightarrow C$ requires two merges and there is no transitive closure of the kind Swoosh formalises ^[47]. The cost of a merge is also structural: it is one page update per affected row, non-atomic, at 3 requests per second. The mean merge in our vault touches 4.6 pages and the largest touches 9. The same merge in SQL is 3 statements inside one transaction, which either all happen or none do.

Table 17 — Entity resolution

Entity rows	1 096
Duplicate pairs	40 (3.65%)
Facts attached to the canonical row	256
Facts attached to the duplicate	105
Subjects whose facts are split across both	39
Facts whose Subject title contradicts their SubjectEntity relation	105 (2.5%)
Aliases claimed by more than one entity	24
Page updates one merge has to issue, mean	4.6
Page updates the largest merge has to issue	9
SQL statements the same merge takes	3

9.5 Where the measurement disappointed us

We predicted that resolving an alias through a comma-joined cell would be expensive, because the only operator that reaches inside the cell is a substring test and substring tests over-match. We built the vault with 57 aliases deliberately constructed as prefixes of others, and measured.

The effect is real and it is small. Across 1 057 alias terms, the substring filter returned 1 159 rows where 1 153 were exact matches — 6 rows fetched and discarded, or 0.5% of the traffic. At this vault's alias density the 1NF violation costs one extra query and a client-side re-split, not a flood of false candidates. We report it because a critique that only prints the measurements that went its way is not a measurement, and because the shape of the risk is worth stating even when the magnitude is not: the over-match grows with the number of aliases sharing a prefix, and nothing in the schema bounds that number.

Table 18 — Alias resolution

Distinct alias terms	1 057
Rows the substring filter returns	1 159
Rows that are exact alias matches	1 153
Rows fetched and discarded by the client	6 (0.5%)
Aliases a prefix of another, by construction	57
Aliases claimed by more than one entity	24
Rows an indexed relation would return	1 153

9.6 The update anomaly

The Boyce-Codd violation of §5.3 has a downstream cost that no amount of careful merging fixes, because it is not about the Facts table at all. A fact is restated across memories — a mean of 2.06 times in our vault, a maximum of 6 — and those restatements are prose. When the fact changes, the Fact row is updated and the prose is not.

Across 1 196 facts whose validity has closed, we found 3 867 memories still asserting the closed value — a mean of 3.23 per fact. For 33 facts, the stale restatements alone would fill a top-ten recall. An agent that retrieves memories rather than facts retrieves the old answer, in a confident sentence somebody wrote, with no marker distinguishing it from the current one.

Table 19 — Stale restatements

Facts whose validity interval has closed	1 196
Memories still asserting the closed value	3 867
Mean stale restatements per closed fact	3.23
Median, where any exist	3
95th percentile, where any exist	8
Facts whose stale copies alone would fill a top-10 recall	33
Times a fact is restated across memories, mean	2.06
Times a fact is restated, maximum	6

10. Retrieval

10.1 The text is not where the filters are

A memory in Lore has a title, properties, and a body of blocks. The body is where the sentence lives. Property filters cannot see blocks, and the search endpoint matches titles. So the only way to search completely over what memories *say*, through the documented API, is to enumerate every memory and fetch each one's children.

At this vault's size that is 11 891 requests and 66.1 min. Ten times larger is 11.0 h; a hundred times larger is 110.1 h. Either engine answers the same question with one statement against an inverted index it already maintains.

Table 20 — The cost of a complete body search

The search endpoint matches titles, and property filters cannot see page blocks, so a complete search over what memories actually say is one list request per hundred rows plus one children request per row.

Vault	Memories	List requests	Body requests	Total	Floor at 3 req/s
This vault	11 773	118	11 773	11 891	66.1 min
Ten times larger	117 730	1 178	117 730	118 908	11.0 h
A hundred times larger	1 177 300	11 773	1 177 300	1 189 073	110.1 h
Either engine, one inverted index	11 773	—	—	1	—

An important qualification. This measures what the *documented* Data API permits, which is what our emulator models. Lore in practice does not do this: it delegates ranking to Notion's own search — a REST endpoint that ranks over titles and bodies, and, behind a feature flag, an internal tool endpoint that is neither publicly documented nor available on every workspace tier. That is a sensible engineering choice and it changes the cost picture materially. It also means Lore owns no index: it cannot pre-filter a semantic query by project or tag (those filters are applied in the client, after ranking), it has no semantic path over Facts at all, and its retrieval quality depends on an endpoint that can be changed or disabled without notice. We could not measure that endpoint and do not claim to have. §14 restates this as the study's main limitation.

10.2 Two indexes, one text, two answers

The workload turned up a result we did not design for. The phrase `timed out` matches nothing in our corpus as a literal substring — the memories say *times out*. SQLite's FTS5, whose default tokeniser does not stem, agrees: 0 rows. PostgreSQL, asked the same phrase, returns 669.

The reason is visible in the parse. `phraseto_tsquery('english', 'timed out')` produces the single lexeme `'time'`: the English configuration stems *timed* to *time* and drops *out* as a stopword, so a two-word phrase query collapses into a one-word one and matches every memory that mentions timing at all. Notion's search endpoint returns nothing for any of the eight phrases, because none of them appears in a title.

None of the three is wrong. A stemmer is a considered position on what “discusses this” means, not a bug. What matters for a memory system is that the position is a choice, and that nothing in any of the three answers says which choice was made. An agent asking the same question of two vaults built on two engines reasons from two different sets of facts.

Table 21 — Three indexes over the same text

Rows are the number of memories each index returns for the same phrase. The literal substring column is the definition the oracle uses; the other three are what the engines think the question means.

Phrase	PostgreSQL parses it as	Substring	FTS5	GIN	Notion search
call graph	'call' <-> 'graph'	677	677	677	0
health endpoint	'health' <-> 'endpoint'	350	350	350	0
on-call rota	'on-cal' <2> 'call' <-> 'rota'	974	974	974	0
error rate	'error' <-> 'rate'	200	200	200	0
nothing unusual	'noth' <-> 'unusu'	379	379	379	0
in-flight requests	'in-flight' <2> 'flight' <-> 'request'	350	350	350	0
deploy manifest	'deploy' <-> 'manifest'	152	152	152	0
timed out	'time'	0	0	669	0

11. The ceiling

Lore's wake-up hook runs before the first response of every session and loads the latest digest, recent memories, active facts and task-matched context. It is the feature that makes the memory ambient rather than requested, and it is also where a memory system's recurring cost lives.

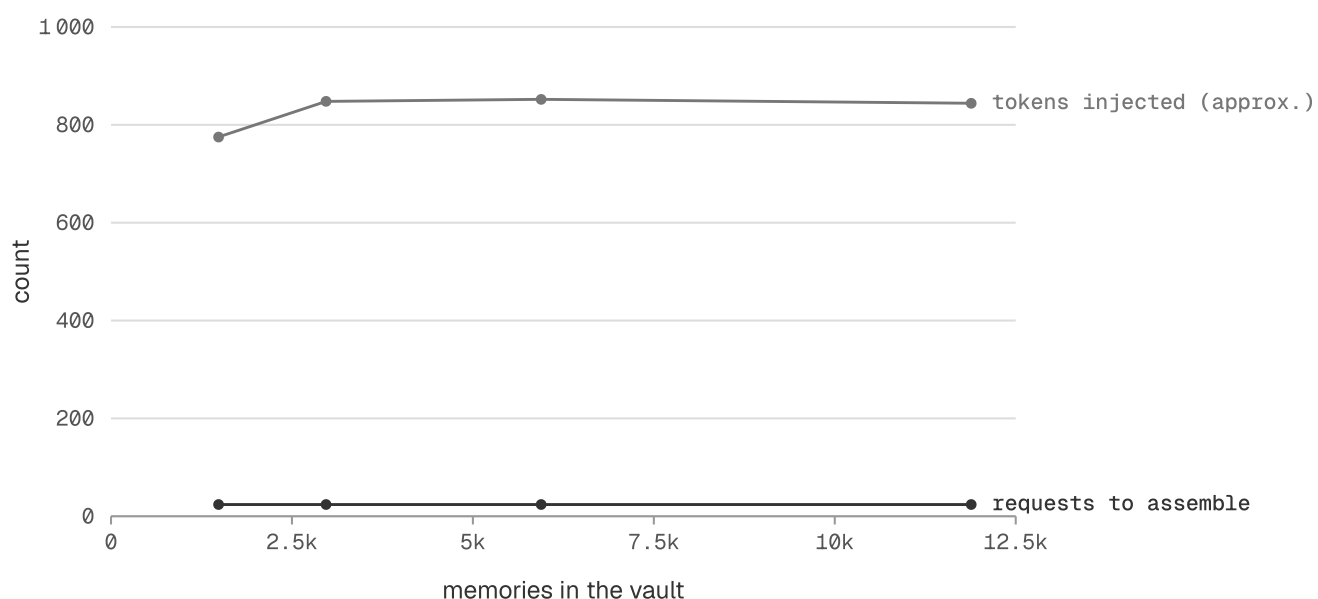
Two quantities scale differently, and separating them is the point of this experiment. The **payload** is capped by construction — 20 memories, 30 facts — so the tokens injected into the model on every session start are nearly flat as the vault grows. The **cost of assembling it** is not, because the queries that produce it run against a growing store, and because the text the hook wants to quote lives in bodies, which is one request per memory.

Table 22 — Wake-up cost as the vault grows

The payload is capped at 20 memories and 30 facts, so the tokens injected are nearly flat. The cost of assembling them is not.

Memories	Facts	Requests	Bytes in	Floor	SQL statements	Tokens injected
1 486	134	24	18.3 kB	8.0 s	4	775
2 971	368	24	20.7 kB	8.0 s	4	848
5 943	1 602	24	20.9 kB	8.0 s	4	852
11 885	4 277	24	23.9 kB	8.0 s	4	844

Figure 9 — Assembly cost against payload size



Requests to assemble the wake-up context, against the tokens that context contains. The second line is flat by design; the first is not.

The conclusion a practitioner should take from this is narrow and useful: a memory system that feels slow at scale is usually not slow because the prompt grew. It is slow because assembling a bounded prompt from an unbounded store costs a number of round trips that grows with the store, and a rate limit turns that number into seconds. Lore mitigates this thoughtfully — a per-session marker debounces the hook, the digest exists precisely to compress the starting context, and the hook can be turned off — but the mitigations are caps, and a cap is where a full scan was found to be infeasible.

12. Explore the evidence

Everything below is computed from the same dataset as the figures above. These are the parts of the argument that are easier to feel than to read.

Explorer 1 — What a question costs

What one question costs

10 classes · 361 questions

provenance Where did this claim come from 40 questions · lore-fact action=query

A fact, its source memory, and the author who wrote it.

σ · ⌵

needs a join

Data API: client-side join

Lore on Notion 1137.75 round trips median 983 · p95 1489



+ 1018 more marks not drawn

1. POST /v1/data_sources/{Facts}/query one request per 100 rows
Predicate equals the predicate. Author lives on Memories, and no filter can reach a property of the page a relation points at, so the predicate is all that can be pushed down

2. GET /v1/pages/{id} one per distinct source memory
The source memory of each candidate fact, to read its Author. Memoised per memory, and still one request each: this is the N+1

3. in the client 1126.2 rows on mean
Drop the archived pages — a page fetched by id comes back even from the trash — and compare Author

SQLite and PostgreSQL 1 statement 1.268 ms in SQLite



engines/sqlite.mjs · provenance

one statement

SELECT f.fact_id
FROM fact f
JOIN memory m ON m.memory_id = f.source_memory_id
WHERE m.author = ? AND f.predicate = ? AND m.archived = 0
ORDER BY f.fact_id

1 138× round trips per statement	6.3 min rate-limit floor at 3 req/s
835.8 kB bytes in, per question	1 126.2 rows examined in the client
100% F1 on the Notion arm	

Every number here is measured, per class, over the questions in that class: round trips are the mean the Notion arm issued, and the rate-limit floor is that mean divided by the 3 requests per second the API documents for one connection — a floor, not a latency, since it is the time the limit alone imposes before any network. Each mark is one round trip, capped at 120 with the remainder written out; the step list is transcribed from

engines/notion.mjs and the statement beside it from engines/sqlite.mjs. Dashed steps cost no request and still cost: they are the rows that crossed the wire so the client could do what the store would not. The Notion arm returns the right answer for every class — 100% F1 on all ten. Nothing here is a correctness argument.

Six of the ten classes are cheap, and the paper says so. What the control is for is the other four: a single join costs 1 138 round trips because Author lives on a page a relation points at; a body search costs 11 892 because a memory's text is blocks and the search endpoint reads titles. Neither is a performance problem that a faster network fixes. Both are the same missing operator.

Explorer 2 — Wake-up cost against vault size

What a bigger vault changes

4 measured sizes · 1486-11885 memories

11 885 memories 4277 facts

measured run

844

tokens injected, Notion arm

8.0 s

rate-limit floor at 3 req/s

3.30

ms

4 statements, SQLite

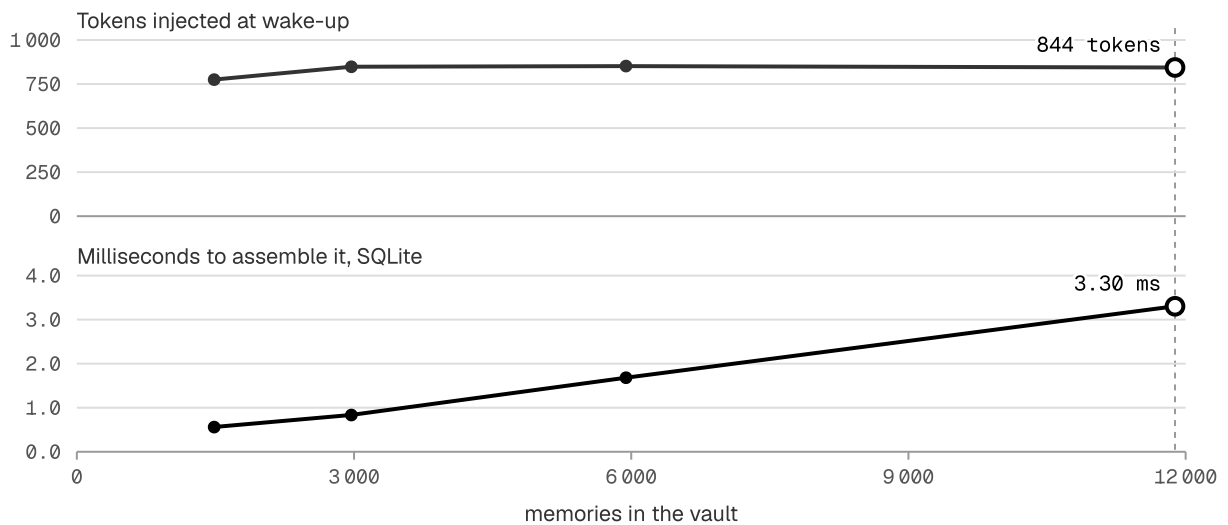
24

round trips to assemble it

23.9 kB

bytes on the wire

At this size the same wake-up costs 2 422× more wall clock on the Notion arm than on SQLite, and the whole of that difference is the rate limit rather than the query: the round trips do not grow with the vault either, they are simply always 24. The token bill, which is the thing a model is actually charged for, is the same on both arms to within 3.0%.



The hook is capped by configuration, not by the vault: 20 recent memories, 30 active facts, 10 open tasks, a 7-day digest window. That is why the payload is flat. Round trips and statement count were identical at all 4 measured sizes, so interpolating them is not a guess; tokens, bytes and milliseconds were not, and a stop between two runs is marked interpolated and names the runs it sits between. Nothing is extrapolated past 11 885 memories, because nothing was measured there. Both panels start at zero.

Across an eight-fold vault the tokens injected at wake-up grow 1.1× and the work to assemble them grows 5.9×. A memory system does not get slow because the prompt got bigger — the prompt is capped, and the model pays the same bill at every size. What grows is the assembly, and on the Notion arm even that is beside the point: the same 24 round trips at 3 requests per second cost 8.0 s before the agent can say its first word, at every vault size measured.

Explorer 3 — The upsert race

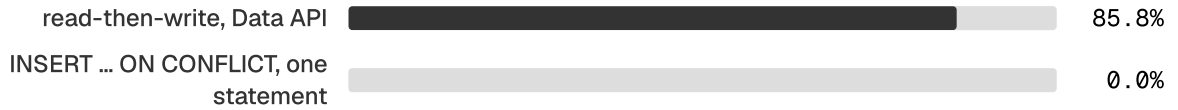
Two writers, one topic key

4 measured writer counts · 200-round headline run

8 writers

measured run

Writes lost, of every write attempted — axis 0 — 100%



Rows created under one topic key, where the invariant allows one — axis 0 — 16 extra



The sweep does not count duplicates on the SQL arm, because the unique index cannot produce one; the 200-round run at 8 writers measured it directly and found 1 row and 0 duplicates.

Read-then-write, step 6 of 6

1. writer A POST /v1/data_sources/{Memories}/query

Topic Key equals runbook/restart-payments-api — 0 results, rows under this key: 0

2. writer B POST /v1/data_sources/{Memories}/query

the same filter, before A has written anything — 0 results, rows under this key: 0

The window opens here, and nothing can close it. The Data API documents no ETag, no If-Match and no conditional write, so B cannot say "create this only if the key is still free".

3. writer A POST /v1/pages

Topic Key set, Revision Count 1 — page up_0_a created, rows under this key: 1

4. writer B POST /v1/pages

the same key, because B probed before A wrote — page up_0_b created, rows under this key: 2

Two rows for one topic key. INV-1 says at most one, and the invariant is enforced by the read-then-write above rather than by the store — so this write is not refused, it is accepted.

5. writer A PATCH /v1/pages/up_0_a

next round: A probed, found the first row, and increments Revision Count to 2 — Revision Count 2, rows under this key: 2

6. writer B PATCH /v1/pages/up_0_a

B probed in the same round and read the same value, so it writes the same number — Revision Count 2, rows under this key: 2

A's increment is gone. Last write wins, and both writers computed their new value from the same stale read. From here every round loses one update per writer beyond the first.

One statement, no steps

tools/exp/integrity.mjs

run by both writers

```
CREATE UNIQUE INDEX ux_topic_key ON memory (topic_key, project_id);

INSERT INTO memory (memory_id, topic_key, project_id, revision_count)
VALUES (?, ?, ?, 1)
ON CONFLICT (topic_key, project_id)
DO UPDATE SET revision_count = revision_count + 1
```

There is nothing to step through. The unique index makes the second writer's insert a conflict rather than a second row, and `DO UPDATE` computes the new revision from the value in the row rather than from a value the client read a moment ago. Both writers run the same statement, in either order, at any width: 1 row, 0 lost updates, 0 duplicates. The invariant is declared where the data is, so it holds for every writer that reaches the data, including the ones nobody wrote the client for.

The sweep runs a fixed schedule of rounds at each of 2, 4, 8, 16 writers, with every round interleaved: all writers probe, then all writers write, which is the worst schedule the protocol admits and the one the API gives no way to exclude. The headline run in the paper is deeper — 200 rounds at 8 writers — and reports 87.1%, above the 85.8% the sweep reports at the same width, because only the first round has nothing yet to lose and its share shrinks as rounds are added. The SQL arm loses 0% at every width measured: the upsert is one statement, so there is no window between the read and the write to interleave anything into. Bars run from zero.

Lore enforces `UNIQUE (topic_key, project_id)` in application code, by reading before writing — which is the only place it can be enforced, because the substrate offers no unique index and no conditional write. The sweep is what that costs under contention. The interleave below is why: not a rare interleaving, but the ordinary one, and the reason a database has had a one-line answer to it since before any of this was written.

13. Discussion

13.1 What the trade actually buys

It would be easy, and wrong, to read this paper as an argument that Lore should have used PostgreSQL. The things Lore gets from Notion are not conveniences.

The memory is legible. A teammate can open the vault, read what the agent believes, and correct it, in a tool they already use, without a client or a query language. No system in the related work offers this. It is the reason a wrong fact gets fixed instead of accumulating.

The permissions already exist. A vault inherits Notion's permission model. A bespoke store would need its own access control replicated from an identity provider, which is a well-known source of leaks. Lore gets a working authorisation story for free, and that is worth more than several of the guarantees it gives up.

There is no infrastructure. No server, no backup policy, no migration window, no on-call rotation for the memory system itself. For a team of five to fifty engineers, the entire operational budget is a tool they already pay for.

Against that: no uniqueness, no referential integrity, no atomicity across pages, no isolation, no joins, no aggregates, and a rate limit that turns every vault-wide operation into a scheduling problem. The honest summary is that Lore trades every guarantee a database declares for legibility, zero infrastructure and inherited permissions, and then re-implements the discarded guarantees as scanners, migrations and filesystem locks. What is unusual, and creditable, is how openly its own source records where the seams are.

13.2 Where the boundary actually is

The measurements suggest a specific and non-obvious boundary. Lore's *retrieval* path is fine: bounded page windows, server-side filters, a capped payload. What degrades is *maintenance and analytics* — the conflict scan, the entity backfill, the debt sweep, the aggregate. Those are the operations that touch every row, and every one of them is $O(n)$ HTTP requests paced by a per-operator token bucket. This is why Lore's caps cluster where they do, and why its own dogfood vault of roughly nine hundred facts already saturates the one server-side aggregate it attempts.

A practitioner reading this should not conclude "do not use Lore". They should conclude: the vault is the system of record because it is legible, and if the vault grows past the point where hygiene passes finish, the answer is a read-side projection — a local relational mirror that carries the indexes, the constraints and the joins, rebuilt from the vault, with the vault remaining the thing humans edit. That is an ordinary architecture with an ordinary name, and nothing in Lore's design forecloses it.

13.3 A note on security

One observation falls outside the database framing but would be irresponsible to omit, because we met it while reading the source. Lore contains a careful prompt-injection containment layer for *upstream* vaults: inherited titles and tags are wrapped in inline code, backticks are doubled, control characters are stripped, and every line is labelled as untrusted so the model weights it down. None of that containment is applied to primary-vault content, which is written by an autosave summariser over conversation text, is editable in the Notion UI by any vault member, and is injected into every teammate's session at wake-up. The mitigation exists and is scoped to the boundary the authors considered external. A shared vault is a same-privilege channel among everyone with write access, and it is worth saying so.

Relatedly, we note that Lore's documented background-hook invocation includes a flag that disables the assistant's permission prompts alongside a tool allowlist, and that the documentation acknowledges the allowlist may need out-of-band configuration. We did not test this and make no claim about exploitability; we record it because it interacts with the previous paragraph.

14. Threats to validity

The Notion arm is an emulator. We hold no workspace and issue no requests. Every restriction it enforces carries a reference page, and it refuses nothing the reference offers — but an emulator cannot capture server-side behaviour that is not documented, and we report counts rather than wall-clock for exactly that reason. A reader who wants the missing number should measure it against a real vault.

We modelled the documented Data API, and Lore uses more than that. This is the most consequential limitation. Lore's search delegates to Notion's own ranking, including an internal endpoint that is undocumented and tier-gated. Our body-search figure of 11 891 requests describes what the documented API permits, not what Lore does. The direction of the error is known — Lore is cheaper than our figure on that class — and the structural point survives it: Lore owns no index, so it cannot pre-filter a semantic query, has no semantic path over Facts, and depends on an endpoint it does not control.

The corpus is synthetic. The vault is generated, which is what makes the ground truth definitional and non-circular, and it is also what makes it not a real team's memory. Restatement rates, alias density and the frequency of ownership changes are parameters we chose. We report them, and the seed reproduces the vault exactly, but a real vault would have a different shape and might have a different alias over-match than the 0.5% we measured.

Some claims about Lore's query surface are inference. We read the schema and a substantial part of the services, but not all of the code paths. Where we say the bitemporal surface is narrower than the schema (§6), or that supersession has no reverse edge, those are readings of the source rather than executions of it, and we have marked them.

PGLite is not a server. PostgreSQL through WebAssembly is the same source tree, planner and type system, but it is not a claim about throughput and we make none. Nothing in this paper depends on a PostgreSQL latency number.

The concurrency schedule is chosen, not observed. We place every read before every write because that is the worst case the protocol admits. Real interleavings are less adversarial and the real loss rate would be lower. The point is not the magnitude but that the API offers no way to make the rate zero.

15. Related work

Agent memory systems. MemGPT framed memory as virtual paging with the model as its own memory manager^[3]; Letta productionised it as shareable memory blocks maintained by asynchronous agents^[4]. Generative Agents introduced the memory stream with recency-, importance- and relevance-weighted retrieval and periodic reflection^[9] — the ancestor of Lore's memory-to-fact promotion and of its confidence decay. MemO is the closest thing in the literature to a DML over agent memory, with explicit ADD, UPDATE, DELETE and NOOP operations^[5]. A-MEM argues the opposite case to Lore's — that a fixed schema limits cross-task adaptability, and that notes should link themselves into an emergent network^[8]. MemOS is the closest in spirit to this paper's framing, treating memory as a managed resource with provenance and version tracking^[21], and MIRIX independently arrives at a six-store typed decomposition^[22].

Temporal knowledge graphs. Zep and its Graphiti engine are the sharpest prior art for Lore's Facts relation: both attach validity windows to subject-predicate-object edges, and Graphiti is explicit that old facts are invalidated rather than deleted, so a vault can be asked what was true at any point in time^{[6][7]}. Lore's schema matches this; §6 argues its query surface reaches less of it.

Retrieval. The dense-retrieval baseline^[29] is complicated by BEIR's finding that BM25 generalises better out of distribution^[32] and by Sciavolino and colleagues' result that dense retrievers underperform badly on entity-centric questions^[33] — which is the regime agent memory lives in, and a good structural argument for Lore's typed Entities table over an undifferentiated vector store. Reciprocal rank fusion^[34] is the fusion rule Lore uses, at the constant its authors recommend.

Databases. The relational model^[39], the entity-relationship model^[41], temporal databases^{[42][43][44]}, slowly changing dimensions^[45], record linkage^{[46][47]}, provenance^{[48][49]}, transactions and isolation^{[50][51][52]}, RDF and named graphs^{[53][55]}, truth maintenance^[56], belief revision^[57], probabilistic databases^[58] and knowledge-graph refinement^[60]. This

paper's claim is not that these are novel but that they are load-bearing, and that a memory system arriving at triples with validity intervals and a confidence has arrived somewhere with fifty years of results in it.

16. Conclusion

Lore is a good schema on a substrate that cannot enforce it. Every one of the five databases is a relation a database designer would recognise, the fact table is bitemporal before most of the field has noticed that it should be, and the maintenance surface — conflict scans, entity merges, dedup backfills — is exactly the surface you build when you know what constraints you would like and cannot declare them.

We measured what that costs. Not correctness: the vault answers 100.0% of a 361-question workload exactly, the same as both engines. It costs requests — 522 686 of them against 361 statements, 48.4 h of rate-limit floor, 11 892× on the worst class — and it costs refusal. Eight agents on one topic key lose 87.1% of their writes. 70 subject-predicate pairs assert two things at once, and the constraint that would have prevented every one of them refuses to be added to the vault at all.

The lesson generalises past Lore. The agent-memory field is converging on triples with validity intervals, entity registries with aliases, provenance edges and confidence scores. That is a database schema. The question every such system has to answer is not which embedding model to use; it is which of the guarantees a database provides it is choosing to do without, and what it will do instead. Lore answers that question more explicitly than most, in comments in its own source. The contribution of this paper is to put the numbers next to the answer.

References

Notion. *Building Shared Memory for AI Agents in Notion*. Notion Engineering Blog, 2026.

www.notion.com/blog/building-shared-memory-for-ai-agents-in-notion

Notion (makenotion). *lore — persistent, shared AI memory backed by Notion for MCP-compatible assistants*. GitHub, MIT licence, commit 95c3558. github.com/makenotion/lore

C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023. arxiv.org/abs/2310.08560

Letta. *Sleep-time agents and memory blocks*. Letta documentation.

docs.letta.com/guides/agents/architectures/sleeptime/

P. Chhikara, D. Khant, S. Aryan, T. Singh, D. Yadav. *MemO: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025. arxiv.org/abs/2504.19413

P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025. arxiv.org/abs/2501.13956

Zep AI. *Graphiti — build temporal context graphs for AI agents*. GitHub, Apache-2.0.

github.com/getzep/graphiti

W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, Y. Zhang. *A-MEM: Agentic Memory for LLM Agents*.

arXiv:2502.12110, 2025. arxiv.org/abs/2502.12110

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. *Generative Agents:*

Interactive Simulacra of Human Behavior. UIST '23, ACM. dl.acm.org/doi/10.1145/3586183.3606763

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023; arXiv:2303.11366. arxiv.org/abs/2303.11366

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv:2305.16291, 2023.

arxiv.org/abs/2305.16291

V. Markovic, L. Obradovic, L. Hajdu, J. Pavlovic. *Optimizing the Interface Between Knowledge*

Graphs and LLMs for Complex Reasoning (Cognee). arXiv:2505.24478, 2025. arxiv.org/abs/2505.24478

LangChain. *LangMem SDK for agent long-term memory*. LangChain Blog, 2025.
www.langchain.com/blog/langmem-sdk-launch

Anthropic. *Effective context engineering for AI agents*. Anthropic Engineering Blog, 2025.
www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Anthropic. *How Claude remembers your project (Claude Code memory)*. Claude Code documentation. code.claude.com/docs/en/memory

OpenAI. *Memory and new controls for ChatGPT*. OpenAI, 2024. openai.com/index/memory-and-new-controls-for-chatgpt/

Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, J.-R. Wen. *A Survey on the Memory Mechanism of Large Language Model based Agents*. ACM TOIS; arXiv:2404.13501.
arxiv.org/abs/2404.13501

Y. Wu, S. Liang, C. Zhang, Y. Wang, Y. Zhang, H. Guo, R. Tang, Y. Liu. *From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs*. arXiv:2504.15965, 2025.
arxiv.org/abs/2504.15965

Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.
arxiv.org/abs/2505.00675

L. Mei et al.. *A Survey of Context Engineering for Large Language Models*. arXiv:2507.13334, 2025.
arxiv.org/abs/2507.13334

Z. Li et al.. *MemOS: A Memory OS for AI System*. arXiv:2507.03724, 2025. arxiv.org/abs/2507.03724

Y. Wang, X. Chen. *MIRIX: Multi-Agent Memory System for LLM-Based Agents*. arXiv:2507.07957, 2025. arxiv.org/abs/2507.07957

A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, Y. Fang. *Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo)*. ACL 2024; arXiv:2402.17753.
arxiv.org/abs/2402.17753

D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, D. Yu. *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. ICLR 2025; arXiv:2410.10813. arxiv.org/abs/2410.10813

Anthropic. *Introducing the Model Context Protocol*. Anthropic News, 25 November 2024.
www.anthropic.com/news/model-context-protocol

Model Context Protocol. *Specification, revision 2024-11-05*. modelcontextprotocol.io.
modelcontextprotocol.io/specification/2024-11-05

Notion. *Notion API reference: request limits, pagination, database query, search*.
developers.notion.com/reference/request-limits

P. Lewis et al.. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020; arXiv:2005.11401. arxiv.org/abs/2005.11401

V. Karpukhin et al.. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020; arXiv:2004.04906. arxiv.org/abs/2004.04906

L. Gao, X. Ma, J. Lin, J. Callan. *Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE)*. ACL 2023; arXiv:2212.10496. arxiv.org/abs/2212.10496

- S. Robertson, H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR 3(4), 2009. dl.acm.org/doi/10.1561/15000000019
- N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS 2021; arXiv:2104.08663. arxiv.org/abs/2104.08663
- C. Sciavolino, Z. Zhong, J. Lee, D. Chen. *Simple Entity-Centric Questions Challenge Dense Retrievers*. EMNLP 2021; arXiv:2109.08535. arxiv.org/abs/2109.08535
- G. V. Cormack, C. L. A. Clarke, S. Büttcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR '09. dl.acm.org/doi/10.1145/1571941.1572114
- S. Bruch, S. Gai, A. Ingber. *An Analysis of Fusion Functions for Hybrid Retrieval*. ACM TOIS 42(1), 2023; arXiv:2210.11934. arxiv.org/abs/2210.11934
- D. Edge et al.. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130, 2024. arxiv.org/abs/2404.16130
- B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS 2024; arXiv:2405.14831. arxiv.org/abs/2405.14831
- B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, Y. Su. *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models*. ICML 2025; arXiv:2502.14802. arxiv.org/abs/2502.14802
- E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. CACM 13(6):377–387, 1970. dl.acm.org/doi/10.1145/362384.362685
- E. F. Codd. *Recent Investigations into Relational Data Base Systems*. IFIP Congress 1974, pp. 1017–1021.
- P. P.-S. Chen. *The Entity-Relationship Model — Toward a Unified View of Data*. ACM TODS 1(1):9–36, 1976. dl.acm.org/doi/10.1145/320434.320440
- R. T. Snodgrass (ed.). *The TSQL2 Temporal Query Language*. Kluwer Academic Publishers, 1995.
- C. S. Jensen, C. E. Dyreson et al.. *The Consensus Glossary of Temporal Database Concepts*. LNCS 1399, Springer, 1998. link.springer.com/chapter/10.1007/BFb0053710
- K. Kulkarni, J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34–43, 2012. dl.acm.org/doi/10.1145/2380776.2380786
- R. Kimball, M. Ross. *The Data Warehouse Toolkit, 3rd edition*. John Wiley & Sons, 2013.
- I. P. Fellegi, A. B. Sunter. *A Theory for Record Linkage*. JASA 64(328):1183–1210, 1969. www.tandfonline.com/doi/abs/10.1080/01621459.1969.10501049
- O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S. E. Whang, J. Widom. *Swoosh: A Generic Approach to Entity Resolution*. The VLDB Journal 18(1):255–276, 2009. link.springer.com/article/10.1007/s00778-008-0098-x
- P. Buneman, S. Khanna, W.-C. Tan. *Why and Where: A Characterization of Data Provenance*. ICDT 2001, LNCS 1973. link.springer.com/chapter/10.1007/3-540-44503-X_20
- J. Cheney, L. Chiticariu, W.-C. Tan. *Provenance in Databases: Why, How, and Where*. Foundations and Trends in Databases 1(4):379–474, 2009. homepages.inf.ed.ac.uk/jcheney/publications/provdbsurvey.pdf

- T. Härder, A. Reuter. *Principles of Transaction-Oriented Database Recovery*. ACM Computing Surveys 15(4):287–317, 1983. dl.acm.org/doi/10.1145/289.291
- J. N. Gray, R. A. Lorie, G. R. Putzolu, I. L. Traiger. *Granularity of Locks and Degrees of Consistency in a Shared Data Base*. Modelling in Data Base Management Systems, North Holland, 1976.
- H. Berenson, P. A. Bernstein, J. Gray, J. Melton, E. O'Neil, P. O'Neil. *A Critique of ANSI SQL Isolation Levels*. SIGMOD '95. dl.acm.org/doi/10.1145/223784.223785
- R. Cyganiak, D. Wood, M. Lanthaler (eds.). *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 2014. www.w3.org/TR/rdf11-concepts/
- S. Harris, A. Seaborne (eds.). *SPARQL 1.1 Query Language*. W3C Recommendation, 2013. www.w3.org/TR/sparql11-query/
- J. J. Carroll, C. Bizer, P. Hayes, P. Stickler. *Named Graphs, Provenance and Trust*. WWW '05, ACM. dl.acm.org/doi/10.1145/1060745.1060835
- J. Doyle. *A Truth Maintenance System*. Artificial Intelligence 12(3):231–272, 1979. www.sciencedirect.com/science/article/abs/pii/0004370279900080
- C. E. Alchourrón, P. Gärdenfors, D. Makinson. *On the Logic of Theory Change: Partial Meet Contraction and Revision Functions*. Journal of Symbolic Logic 50(2):510–530, 1985.
- D. Suciu, D. Olteanu, C. Ré, C. Koch. *Probabilistic Databases*. Morgan & Claypool, 2011.
- X. L. Dong et al.. *Knowledge Vault: A Web-Scale Approach to Probabilistic Knowledge Fusion*. KDD '14, pp. 601–610. www.cs.ubc.ca/~murphyk/papers/kv-kdd14.pdf
- H. Paulheim. *Knowledge Graph Refinement: A Survey of Approaches and Evaluation Methods*. Semantic Web 8(3):489–508, 2017. dl.acm.org/doi/10.3233/SW-160218

Appendices

A. What we read

Every claim this paper makes about Lore's implementation was read from one of these files, in a clone at commit 95c3558.

<https://github.com/makenotion/lore> at 95c3558, 2026-08-04.

File	What it settled
<code>src/notion/schema.ts</code>	the five property catalogues, verbatim
<code>src/core/entity.ts</code>	alias serialisation and resolution
<code>src/core/entity-merge.ts</code>	what a merge does and does not touch
<code>src/core/memory-topic-key.ts</code>	the upsert protocol and its admitted race
<code>src/core/decay.ts</code>	the confidence algebra
<code>src/policy/confidence.ts</code>	the confidence constants
<code>src/cli/commands/conflicts.ts</code>	the conflict scan and its cap

B. The DDL both engines execute

Generated from the same declaration as the diagram in §4. The PostgreSQL dialect is shown; the SQLite dialect differs only in type names and in the constraints SQLite cannot express.

```

CREATE TABLE project (
  project_id TEXT NOT NULL,
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('project', 'person', 'agent')),
  path TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('active', 'archived')),
  description TEXT,
  PRIMARY KEY (project_id)
);

CREATE TABLE topic (
  topic_id TEXT NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  project_id TEXT,
  PRIMARY KEY (topic_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id)
);

CREATE TABLE memory (
  memory_id TEXT NOT NULL,
  title TEXT NOT NULL,
  kind TEXT NOT NULL CHECK (kind IN ('note', 'decision', 'incident', 'runbook', 'postmortem', 'policy', 'state',
'operational', 'task', 'procedure')),
  source TEXT NOT NULL CHECK (source IN ('conversation', 'autosave_learning', 'file', 'manual', 'agent_diary', 'digest')),
  status TEXT NOT NULL CHECK (status IN ('informational', 'proposed', 'accepted', 'superseded', 'deprecated', 'rejected')),
  task_state TEXT,
  blocked_by TEXT,
  entity_text TEXT,
  author TEXT NOT NULL,
  agent TEXT NOT NULL,
  session TEXT,
  topic_key TEXT,
  revision_count BIGINT NOT NULL,
  keywords TEXT,
  synopsis TEXT,
  compare_notes TEXT,
  promotion_source_key TEXT,
  review_by BIGINT,
  done_at BIGINT,
  decided_at BIGINT,
  last_referenced_at BIGINT,
  alternatives TEXT,
  consequences TEXT,
  scope_kind TEXT,
  scope_key TEXT,
  audience TEXT,
  lifetime TEXT,
  expires_at BIGINT,
  expires_on TEXT,
  pinned BIGINT,
  pinned_priority BIGINT,
  mutability TEXT,
  supersedes_id TEXT,
  affects_id TEXT,
  compared_with_id TEXT,
  body TEXT NOT NULL,
  created_time BIGINT NOT NULL,
  last_edited_time BIGINT NOT NULL,
  archived BIGINT NOT NULL,
  PRIMARY KEY (memory_id),
  FOREIGN KEY (supersedes_id) REFERENCES memory(memory_id),
  FOREIGN KEY (affects_id) REFERENCES memory(memory_id),
  FOREIGN KEY (compared_with_id) REFERENCES memory(memory_id)
);

CREATE TABLE memory_project (
  memory_id TEXT NOT NULL,
  project_id TEXT NOT NULL,
  PRIMARY KEY (memory_id, project_id),
  FOREIGN KEY (memory_id) REFERENCES memory(memory_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id)
);

CREATE TABLE memory_topic (
  memory_id TEXT NOT NULL,
  topic_id TEXT NOT NULL,
  PRIMARY KEY (memory_id, topic_id),
  FOREIGN KEY (memory_id) REFERENCES memory(memory_id),
  FOREIGN KEY (topic_id) REFERENCES topic(topic_id)
);

```

```

CREATE TABLE memory_tag (
  memory_id TEXT NOT NULL,
  tag TEXT NOT NULL,
  PRIMARY KEY (memory_id, tag),
  FOREIGN KEY (memory_id) REFERENCES memory(memory_id)
);

CREATE TABLE entity (
  entity_id TEXT NOT NULL,
  name TEXT NOT NULL,
  aliases_raw TEXT,
  kind TEXT,
  description TEXT,
  project_id TEXT,
  source_memory_id TEXT,
  PRIMARY KEY (entity_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (source_memory_id) REFERENCES memory(memory_id)
);

CREATE TABLE entity_alias (
  entity_id TEXT NOT NULL,
  alias TEXT NOT NULL,
  PRIMARY KEY (entity_id, alias),
  FOREIGN KEY (entity_id) REFERENCES entity(entity_id)
);

CREATE TABLE fact (
  fact_id TEXT NOT NULL,
  subject TEXT NOT NULL,
  predicate TEXT NOT NULL,
  object TEXT NOT NULL,
  valid_from BIGINT,
  valid_until BIGINT,
  observed_at BIGINT,
  invalidated_at BIGINT,
  invalidated_by TEXT,
  confidence TEXT NOT NULL CHECK (confidence IN ('certain', 'likely', 'speculative')),
  confidence_score DOUBLE PRECISION,
  last_referenced_at BIGINT,
  review_by BIGINT,
  dedup_key TEXT,
  subject_key TEXT,
  project_id TEXT,
  source_memory_id TEXT,
  subject_entity_id TEXT,
  object_entity_id TEXT,
  scope_kind TEXT,
  scope_key TEXT,
  lifetime TEXT,
  expires_at BIGINT,
  created_time BIGINT NOT NULL,
  PRIMARY KEY (fact_id),
  FOREIGN KEY (invalidated_by) REFERENCES memory(memory_id),
  FOREIGN KEY (project_id) REFERENCES project(project_id),
  FOREIGN KEY (source_memory_id) REFERENCES memory(memory_id),
  FOREIGN KEY (subject_entity_id) REFERENCES entity(entity_id),
  FOREIGN KEY (object_entity_id) REFERENCES entity(entity_id)
);

```

C. The constraints Notion cannot express

```
-- INV-1
CREATE UNIQUE INDEX ux_memory_topic_key ON memory_topic_key (topic_key, project_id);

-- INV-5
CREATE UNIQUE INDEX ux_alias ON entity_alias (alias);

-- INV-7: The expression index DedupKey and SubjectKey exist to stand in for.
CREATE UNIQUE INDEX ux_fact_dedup ON fact (lower(subject), predicate, lower(object));

-- INV-6
ALTER TABLE fact ADD CONSTRAINT ck_validity CHECK (valid_until IS NULL OR valid_from <= valid_until);

-- INV-3: A temporal primary key. SQLite has no exclusion constraints; Notion has no constraints.
ALTER TABLE fact ADD CONSTRAINT ex_fact_validity
  EXCLUDE USING gist (
    subject_entity_id WITH =,
    predicate WITH =,
    int8range(COALESCE(valid_from, -999999), COALESCE(valid_until, 999999), '[]') WITH &&
  ) WHERE (predicate IN ('owned_by','is_a','created_by') AND subject_entity_id IS NOT NULL);
```

D. Toolchain and reproduction

Captured	2026-08-22 07:15:26 UTC
Duration	1.6 min
Repository commit	d413980
Node	v26.1.0
SQLite	node:sqlite (bundled with v26.1.0)
PGLite	0.5.4
Machine	13th Gen Intel(R) Core(TM) i5-13450HX, 16 cores, 31.7 GB, win32/x64
Timed runs per question	3
Subject	https://github.com/makenotion/lore @ 95c3558 (1.0.0, MIT, Notion Labs, Inc.)

The whole dataset regenerates with `npm install && npm run paper`. There is no database server to install and no API key to set: SQLite comes from Node’s built-in `node:sqlite`, and PostgreSQL runs through PGLite as WebAssembly. The vault is deterministic in its seed, so a rerun reproduces every number on this page.

Harshit Khemani · Kush Ahuja · Madhav Bassi · Kushagra Agrawal · submitted to Dr. Poonam Sangwan

Measured 22 August 2026 on 13th Gen Intel(R) Core(TM) i5-13450HX. Subject read at 95c3558. Lore is © Notion Labs, Inc., used under the MIT licence; this paper is coursework and is not affiliated with Notion.